

Signatures with Tight Adaptive Corruptions from Search Assumptions

Keitaro Hashimoto¹, Wakaha Ogata², and Yusuke Sakai¹

¹ National Institute of Advanced Industrial Science and Technology (AIST)
{keitaro.hashimoto, yusuke.sakai}@aist.go.jp

² Institute of Science Tokyo
ogata.w.4ea4@m.isct.ac.jp

Abstract. In this paper, we show that tightly secure signature schemes in the multi-user setting with adaptive corruptions can be achieved from various weak assumptions, such as classical discrete logarithm, RSA, factoring, and even the post-quantum group action discrete logarithm assumption, at the cost of signature length. To this end, we introduce a new framework to construct a signature scheme. Our construction is a combination of an identification scheme with multiple secret keys per public key (with additional specific features) and (randomized) Fischlin transformation. From the straight-line extractability of the transformation, intuitively unforgeability can be tightly reduced to the soundness of an identification scheme in the programmable random oracle model (PROM). Our careful security proof succeeds in a tight reduction in the non-programmable random oracle model (NPRO). As a complementary study, we analyze the multi-user security of hash-based signatures. Although hash-based signatures seem to be tightly secure in the multi-user setting with adaptive corruptions (at the cost of signature lengths similar to our construction), their security has not been closely analyzed. We show the tight security based on collision resistance of the underlying hash function by modeling the pseudo-random function for key generation as a (programmable) random oracle. By using a hash function whose security is tightly reduced to other assumptions, such as the discrete logarithm problem, we also obtain a tightly secure signature based on them. However, the concrete signature size is larger than our proposal based on an identification scheme, and the security reduction is in the PROM in contrast to our new framework.

Keywords: Signature, Multi-user setting with corruption, Tight security

1 Introduction

1.1 Background

Digital signature. Signature schemes are fundamental cryptographic tools to authenticate the sender of messages. The basic security notion for signature schemes is existential unforgeability against chosen message attacks (UF-CMA) [GMR88]. It guarantees that an efficient adversary, given a single verification key, cannot forge a valid signature for any new message under that key. UF-CMA security considers a single-user setting, but in real-world scenarios, multiple users possess individual signing keys derived from a common public parameter, and some of these keys may occasionally be compromised. To address a more realistic context, Bader et al. [BHJ⁺15] defined the notion of UF-CMA *in the multi-user setting with adaptive corruptions* (MU-UF-CMA-C). This framework extends UF-CMA by allowing adversaries to obtain multiple users' verification keys and adaptively corrupt users to access their secret signing keys. Notably, MU-UF-CMA-C security precisely captures the requirements of practical applications that rely on digital signatures, such as authenticated key exchange [BHJ⁺15, PQR21, PQR22, HJK⁺21] and identity-based signature [LPLL20].

Tight security. The security of cryptographic primitives is proved by constructing a reduction algorithm that transforms an efficient adversary who breaks the security of the scheme (e.g., the UF-CMA security of the signature scheme) into an algorithm that solves an assumed-to-be-hard computational problem (e.g., the discrete logarithm problem). In general, the success probability of the reduction and that of the adversary

have some gap called *reduction loss*, representing the theoretical difference between the hardness of breaking the scheme’s security and the hardness of solving the computational problem. If the reduction has almost the same success probability as the adversary, the reduction is said to be *tight*, and the scheme is said to be tightly secure.³ This means the primitive’s security is independent of the adversary’s behavior (e.g., the number of hash computations and the number of signing queries) and the number of users in the system. Tight security is important in both theory and practice. A tight reduction reveals the connection between the hardness of breaking the security and that of solving computational problems. Since computational problem hardness is generally well-studied, we can easily understand the security level of tightly secure schemes. In addition, tightly secure schemes allow for optimal parameter selection that meets the desired security level based on cryptanalysis against underlying computational problems. As a result, data size (e.g., key and signature sizes) and computation cost (e.g., signing and verification) can be minimized, resulting in more efficient schemes than non-tight schemes with large parameter sets, considering the reduction loss.

Tight MU-UF-CMA-C secure signature. It is known that EU-CMA security implies MU-UF-CMA-C security, albeit with a reduction loss proportional to the number of users. In other words, such signature schemes require a larger parameter to be selected to satisfy a sufficient security level to account for the reduction loss. To mitigate this loss, researchers have focused on developing *tightly* MU-UF-CMA-C-secure signature schemes, where the security remains independent of the number of users and other factors such as the numbers of signing and random oracle queries [Bad14,BHJ⁺15,GJ18,WLG⁺19,DGJL21], summarized in Table 1. We notice that they are based on decisional assumptions (e.g., (group action) DDH, DLIN, SXDH, ϕ -hiding) or interactive search assumptions (e.g., one-more CDH), and no tightly MU-UF-CMA-C-secure signature scheme has been constructed under static search assumptions (e.g., CDH or DL).

On the other hand, hash-based signatures seem to be tightly secure in the multi-user setting with adaptive corruptions. If so, their security can also be tightly reduced to a search assumption by using a hash function whose security is tightly reduced to a search problem. However, the multi-user security with adaptive corruption of hash-based signatures has not been closely analyzed.

1.2 Our Contributions

In this paper, we provide a new framework to achieve a tightly secure signature scheme based on search problems, including post-quantum problems (e.g., GADL). The security of our schemes is independent of the number of users, signing queries, and random oracle (RO) queries, and forging a signature is provably as hard as solving static search problems.

Our signature is a generic construction based on a certain kind of identification scheme that allows multiple secret keys per public key, such as Okamoto identification scheme [Oka93] and the Parallel-OR identification scheme [CDS94]. To tightly convert such an identification scheme into a signature scheme, we utilize randomized Fischlin transformation introduced by Kondi and shelat [Ks22], which is a variant of Fischlin’s transformation [Fis05]. It is known that randomized Fischlin transformation does not satisfy the zero-knowledge property if the challenge space is polynomial size, but the ZK is satisfied by using exponential size challenge space (in the programmable random oracle model) [KRW24]. In contrast, the tight security of our signature scheme is proven in the *non-programmable* ROM *without enlarging* the challenge space.

By instantiating the underlying identification scheme appropriately, we obtain tightly MU-UF-CMA-C secure signature schemes based on various static search assumptions such as classical DL, RSA, or factoring assumptions and post-quantum group action DL (GADL) assumption, as shown in Table 1.

It is worth noting that we obtain tightly MU-UF-CMA-C secure signatures from computational assumptions that are not random self-reducible (e.g., RSA assumption). Further, under the weaker assumption, our group action-based scheme achieves a signature length comparable to that of the scheme by Pan and Wagner [PW22].

³ In general, negligibly small statistical terms are ignored even if the terms depend on some parameters. For example, if the reduction’s success probability is expressed as $\text{Adv}(\lambda) + Q/2^\lambda$ where λ is the security parameter, $\text{Adv}(\lambda)$ is the advantage for some computational problem, and Q is the number of random oracle queries, we say the reduction is tight.

Table 1: Existing tightly secure signatures in the multi-user setting with corruptions and our result. The column “Settings” indicates whether pairings, Common Reference String (CRS), Programmable Random Oracle (PRO), and Non-Programmable Random Oracle (NPRO) are used. The column “SUF” indicates whether the scheme is strongly unforgeable. λ denotes the security parameter, t denotes the bit-length of the challenge space, and ρ denotes the number of repetitions. Example settings of (ρ, t) are $(\rho, t) = (32, 9), (22, 11), (16, 13)$ when $\lambda = 128$ (cf. Section 6). $|X|$ denotes the bit-length of the elements in the set X .

(a) Group-based schemes. Let $\mathbb{G}$ be a multiplicative group with order q .

Scheme	Public key	Signature	Assumptions	Settings	SUF?
Bader [Bad14]	$ \mathbb{G} $	$6 \mathbb{G} $	SXDH	Pairing, PRO	—
BHJKL [BHJ ⁺ 15]	$\mathcal{O}(\lambda) \mathbb{G} $	$\mathcal{O}(\lambda) \mathbb{G} $	DLIN	Pairing, CRS	—
GJ [GJ18]	$2 \mathbb{G} $	$2 \mathbb{G} + 4 \mathbb{Z}_q + 2\lambda$	CDH & DDH	PRO	—
WLSZ [WLG ⁺ 19]	$ \mathbb{G} $	$2 \mathbb{G} + 1$	OM-CDH	Pairing, PRO	—
DGJL [DGJL21, Sec. 5.1]	$4 \mathbb{G} $	$3 \mathbb{Z}_q $	DDH	NPRO	✓
Ours (based on [Oka93])	$ \mathbb{G} $	$\rho(2 \mathbb{Z}_q + t)$	DL	NPRO	✓

(b) Factoring-based and RSA-based schemes. Let N be an integer such that $N = pq$ for some primes p and q .

Scheme	Public key	Signature	Assumptions	Settings	SUF?
DGJL [DGJL21, Sec. 5.2]	$2 \mathbb{Z}_N $	$2 \mathbb{Z}_N + \lambda/4$	ϕ -hiding	NPRO	✓
Ours (based on [Oka93])	$ \mathbb{Z}_N $	$\rho(2 \mathbb{Z}_N + t)$	RSA	NPRO	✓
Ours (based on [FF02])	$ \mathbb{Z}_N $	$\rho(2 \mathbb{Z}_N + 2t + \text{poly}(\lambda))$	FACT	NPRO	✓

(c) Group action-based schemes. Let $\mathcal{G}$ be a group that acts on a set $\mathcal{E}$ (i.e., there exists a group action $\star : \mathcal{G} \times \mathcal{E} \rightarrow \mathcal{E}$).

Scheme	Public key	Signature	Assumptions	Settings	SUF?
PW [PW22, Sec. 4.2]	$4 \mathcal{E} $	$2\lambda(\mathcal{G} + 1)$	GADDH	NPRO	—
Ours (based on [Cou06,BKV19,Sto12])	$2 \mathcal{E} $	$2\rho t(\mathcal{G} + 1)$	GADL	NPRO	✓

(d) Hash-based schemes. $R_{H_{\mathcal{E}}}$ denotes the range of the hash function.

Scheme	Public key	Signature	Assumptions	Settings	SUF?
Merkle [Mer79] (cf. Sec. 5)	2λ	$\mathcal{O}(\lambda) R_{H_{\mathcal{E}}} $	CR	PRO	✓

As a complementary study, we analyze the MU-UF-CMA-C security of hash-based signature schemes. In particular, we show that the MU-UF-CMA-C security of the Merkle signature scheme (MSS)[Mer79,Mer90] with the hyper-tree extension[BDK⁺07], instantiated with the Lamport one-time signature scheme can be tightly reduced to collision resistance of the underlying hash function in the random oracle model⁴. In contrast to our main contribution, the random oracle must be programmable in this scheme. Furthermore, the concrete signature size exceeds that of our signature scheme.

1.3 Technical Overview

This section provides an intuition of our signature scheme based on identification schemes.

Our construction starts with a specialized identification scheme with multiple secret keys per public key and a property called “second-key recovery resistance.” This property means that for a given key pair (pk, sk) , computing another valid secret key sk' with respect to pk can be reduced to breaking a search assumption. To tightly convert such an identification scheme into a signature scheme, we utilize the randomized Fischlin

⁴ In our analysis, not the hash function but the pseudo-random function used to generate many Lamport signing keys is modeled as the random oracle.

transformation [Ks22]. In this transformation, the prover, given a statement x and a witness w , first generates a commitment of the underlying Sigma protocol com . Then, the prover searches a challenge ch in a random order such that the hash value $h = H(\text{com}, \text{ch}, \text{resp})$ is the all-zero string where resp is the response under com and ch . The proof is the transcript $(\text{com}, \text{ch}, \text{resp})$, i.e., a valid transcript of the Sigma protocol that yields the all-zero hash value.⁵ This mechanism realizes straight-line extraction in the ROM, since the prover is imposed to query the RO for multiple valid transcripts that share the same commitment until getting the all-zero answer. That is, the extractor can obtain sufficient valid transcripts to extract a witness by observing the RO queries.

Now, let us explain how the reduction $\mathcal{B}$ from the MU-UF-CMA-C security of the obtained scheme to a search assumption works. $\mathcal{B}$ first generates key pairs $(\text{sk}_i, \text{pk}_i)$ for all users by itself, and runs MU-UF-CMA-C adversary $\mathcal{A}$. Indeed, $\mathcal{B}$ can simulate the signing oracle and the corruption oracle perfectly using the secret keys sk_i . If $\mathcal{A}$ outputs a valid signature under a public key pk_i , $\mathcal{B}$ can extract a secret key sk'_i corresponding to pk_i due to the straight-line extractability of randomized Fischlin transformation. If the extracted key sk'_i is different from sk_i generated by $\mathcal{B}$, $\mathcal{B}$ succeeds in breaking the search assumption from the second-key recovery resistance of the underlying identification scheme. Now, we hope $\text{sk}'_i \neq \text{sk}_i$ holds with high probability. Of course, $\mathcal{A}$ is disallowed to get sk_i from the corruption oracle, but our concern is that $\mathcal{A}$ might be able to get some information about sk_i from the answers of the signing oracle (i.e., signatures) simulated with sk_i and make $\text{sk}'_i = \text{sk}_i$. Roughly, signatures of our scheme are non-interactive zero-knowledge (NIZK) proofs obtained by randomized Fischlin transformation, but, unfortunately, we cannot easily conclude that signatures give no information about the used key sk_i , since the transformation does not necessarily provide *perfect* zero-knowledge property. Instead, we prove that $\mathcal{A}$ actually has no information about sk_i unless $\mathcal{A}$ issues special queries to the random oracle, and that it is infeasible for PPT adversaries to make such special queries.

Interestingly, the tight security of our signature scheme is shown in the *non-programmable* ROM, although the transformation guarantees (possibly computational) zero-knowledge property in the *programmable* ROM. This is because the programmability of the RO is only used to show $\mathcal{A}$ has no information about sk_i in our security proof, and the reduction $\mathcal{B}$ does not need to program the RO.

1.4 Future Work and Open Problems

We leave the following interesting future works and open problems.

First, the signatures of both of our new construction and hash-based schemes consist of the number of elements proportional to the security parameter λ , i.e., the signature bit length is $\Theta(\lambda^2)$, taking into account that the length of each element should be $\Theta(\lambda)$ for λ -bit security. Constructing a tight MU-UF-CMA-C secure signature with a $o(\lambda^2)$ signature size from search assumptions is an important future work and an interesting open problem.

Second, the security of the randomized Fischlin transform in the quantum ROM (QROM) has not been shown, so the tight security of our construction is not guaranteed in the QROM either. A promising alternative approach to achieve a tight MU-UF-CMA-C signature from post-quantum search assumption (e.g., GADL) in the QROM is the use of Pass's transformation [Pas03], i.e., a Sigma protocol is first transformed into a commit-then-open type interactive protocol and then it is converted to a non-interactive protocol using Fiat-Shamir (FS) transformation [FS87], since the result by Don et al. [DFMS22] allows us to convert commit-then-open type protocols into NIZK via FS transformation in the QROM tightly. A drawback of Pass's transformation is its inefficiency. The proof length is $\mathcal{O}(\lambda)$ times longer than the total length of a transcript in the underlying Sigma protocol. However, it is comparable to our group action-based scheme in which the signature consists of $\mathcal{O}(\lambda)$ group elements.

⁵ To balance the soundness error and the efficiency, in the actual protocol, the prover does this process in parallel, but this is not important for the moment.

2 Preliminaries

This section reviews basic notations and definitions of cryptographic primitives used in this paper.

2.1 Notations

$\lambda \in \mathbb{N}$ denotes a security parameter. $\oplus$ denotes the bit-wise exclusive-or operation. $\text{poly}(\cdot)$ and $\text{negl}(\cdot)$ are any polynomial function and negligible function, respectively. e denotes the base of the natural logarithm (i.e., Napier's constant). For $n \in \mathbb{N}$, we define $[n] := \{1, 2, \dots, n\}$ as the set of the first n natural numbers. For a finite set S , we use $s \leftarrow \$ S$ to denote the uniformly random sampling of an element s from S . A probabilistic algorithm $\mathcal{A}$ is said to be PPT (probabilistic polynomial time) if its running time $T_{\mathcal{A}}$ is bounded by a polynomial in its input size. The notation $y \leftarrow \mathcal{A}(x)$ means that the variable y is assigned to the output of the algorithm $\mathcal{A}$ on input x . We write $y \in \mathcal{A}(x)$ to state that y is a possible output of $\mathcal{A}$ on input x .

2.2 Signature Schemes

We recall the syntax and security notions of signature schemes.

Definition 1 (Signature scheme). *A signature scheme SIG is a tuple of the following algorithms.*

- $\text{Setup}(1^\lambda) \rightarrow \text{par}$: On input the security parameter 1^λ , the setup algorithm outputs a public parameter par . We assume the following algorithms implicitly take par as input.
- $\text{KGen}(\text{par}) \rightarrow (\text{svk}, \text{ssk})$: On input a public parameter par , the key generation algorithm outputs a public key svk and a secret key ssk .
- $\text{Sign}(\text{ssk}, m) \rightarrow \sigma$: On input a secret key ssk and a message m , the signing algorithm outputs a signature σ .
- $\text{Vf}(\text{svk}, m, \sigma) \rightarrow 1/0$: On input a public key svk , a message m , and a signature σ , the verification algorithm outputs 0 or 1.

Definition 2 (Correctness). *We say that a signature scheme SIG is $(1 - \beta)$ -correct if for any $\lambda \in \mathbb{N}$, any $\text{par} \in \text{Setup}(1^\lambda)$, any key pair $(\text{svk}, \text{ssk}) \in \text{KGen}(\text{par})$, and any message m , it holds that*

$$\Pr[\text{Vf}(\text{svk}, m, \text{Sign}(\text{ssk}, m)) = 1] \geq 1 - \beta.$$

In this work, we focus on strong existential unforgeability against adaptive chosen-message attacks in the multi-user setting with adaptive corruptions [DGJL21, Definition 2].⁶ We call it MU-SUF-CMA-C security in short.

Definition 3 (N -MU-SUF-CMA-C Security [DGJL21]). *Let $N = \text{poly}(\lambda)$ be a natural number. We say a signature scheme SIG is N -MU-SUF-CMA-C secure if for any PPT adversary $\mathcal{A}$, it holds that*

$$\text{Adv}_{\mathcal{A}, \text{SIG}}^{\text{MU-SUF-CMA-C}}(\lambda) := \Pr[N\text{-MU-SUF-CMA-C} \Rightarrow 1] \leq \text{negl}(\lambda),$$

where the game N -MU-SUF-CMA-C is depicted in Figure 1.

Similarly, we define one-time N -MU-SUF-CMA-C security by limiting the number of OSign queries for each signing key to (at most) one.

⁶ As mentioned in [DGJL21], strong unforgeability is useful for constructing cryptographic protocols such as authenticated key exchange.

$N\text{-MU-SUF-CMA-C:}$	$\text{OSign}(i, m)$
1 : $L_{\text{corr}}, L_{\text{sig}} \leftarrow \emptyset$	1 : if $i \in L_{\text{corr}}$ then
2 : $\text{par} \leftarrow \text{Setup}(1^\lambda)$	2 : return $\perp$
3 : foreach $i \in [N]$ do	3 : $\sigma \leftarrow \text{Sign}(\text{ssk}_i, m)$
4 : $(\text{svk}_i, \text{ssk}_i) \leftarrow \text{KGen}(\text{par})$	4 : $L_{\text{sig}} := L_{\text{sig}} \cup \{(i, m, \sigma)\}$
5 : $\text{SVK} := \{\text{svk}_i\}_{i \in [N]}$	5 : return σ
6 : $(i^*, m^*, \sigma^*) \leftarrow \mathcal{A}^{\text{OSign}, \text{OCorr}}(\text{par}, \text{SVK})$	$\text{OCorr}(i)$
7 : if $i^* \in L_{\text{corr}}$ then return 0	1 : $L_{\text{corr}} := L_{\text{corr}} \cup \{i\}$
8 : if $(i^*, m^*, \sigma^*) \in L_{\text{sig}}$ then	2 : return ssk_i
9 : return 0	
10 : $\text{ok} := \text{Vf}(\text{svk}_{i^*}, m^*, \sigma^*)$	
11 : return ok	

Fig. 1: Security game for signature scheme.

2.3 Canonical Identification Schemes

We follow the syntax of canonical identification schemes in [KMP16].

Definition 4 (Canonical Identification Schemes). A canonical identification scheme ID is a tuple of the following four algorithms.

- $\text{ISetup}(1^\lambda) \rightarrow \text{par}$: The setup algorithm takes the security parameter 1^λ and outputs a public parameter par . We assume that par defines the challenge space ChSet and the following algorithms implicitly take par as input.
- $\text{IGen}(\text{par}) \rightarrow (\text{pk}, \text{sk})$: The key generation algorithm takes a public parameter par as input and outputs public and secret keys (pk, sk) . We assume the secret key sk is chosen uniformly and randomly from the secret key space.
- $\text{P} = (\text{P}_1, \text{P}_2)$: The prover algorithm is split into two algorithms. P_1 takes as input a key pair (pk, sk) and returns a commitment com and a state st ; P_2 takes as input a challenge ch and a state st , and returns a response resp .
- $\text{V}(\text{pk}, \text{com}, \text{ch}, \text{resp}) \rightarrow 1/0$: The verifier algorithm takes a public key pk and a conversation transcript $(\text{com}, \text{ch}, \text{resp})$ as input and outputs 1 or 0.

Let $K := |\{\text{sk} : (\text{pk}, \text{sk}) \in \text{IGen}(\text{par})\}|$ denote the number of valid secret keys w.r.t. a public key pk . (We assume that every public key corresponds to the same number of valid secret keys, since all known schemes satisfy this property.) We say that an identification scheme ID has K -multiple secret keys if each pk has K secret keys. When $K = 1$, we say that ID has a single secret key.

We require that identification schemes ID satisfy the following properties.

Definition 5 (Correctness). We say that ID is correct if for all $\lambda \in \mathbb{N}$, all $\text{par} \in \text{ISetup}(1^\lambda)$, all $(\text{pk}, \text{sk}) \in \text{IGen}(\text{par})$, all $(\text{com}, \text{st}) \in \text{P}_1(\text{pk}, \text{sk})$, all $\text{ch} \in \text{ChSet}$ and all $\text{resp} \in \text{P}_2(\text{ch}, \text{st})$, we have $\text{V}(\text{pk}, \text{com}, \text{ch}, \text{resp}) = 1$.

We say a transcript $(\text{com}, \text{ch}, \text{resp})$ is valid w.r.t. pk if $\text{V}(\text{pk}, \text{com}, \text{ch}, \text{resp}) = 1$.

Definition 6 (Key Verifiability [PW22]). We say that ID is key verifiable if there exists a deterministic polynomial time algorithm VerKey such that for all $\lambda \in \mathbb{N}$, all $\text{par} \in \text{ISetup}(1^\lambda)$ and any (pk, sk) ,

$$\text{VerKey}(\text{par}, \text{pk}, \text{sk}) = 1 \iff (\text{pk}, \text{sk}) \in \text{IGen}(\text{par}).$$

Definition 7 (Min-Entropy of Commitments [KMP16]). We say that ID has κ -bits of commitment min-entropy, if for all $(pk, sk) \in \text{IGen}(\text{par})$, the commitment generated by the prover algorithm is chosen from a distribution with at least κ -bits of commitment min-entropy. That is, for all strings com' , we have $\Pr[\text{com}' = \text{com}] \leq 2^{-\kappa}$ if $(\text{com}, *) \leftarrow P_1(pk, sk)$ was honestly generated by the prover.

Given ID as above, we define transcript generation algorithm Tran as follows:

Tran(pk, sk, ch)
1 : $(\text{com}, \text{st}) \leftarrow P_1(pk, sk)$
2 : $\text{resp} \leftarrow P_2(\text{ch}, \text{st})$
3 : return $(\text{com}, \text{ch}, \text{resp})$

Definition 8 (Special Honest-Verifier Zero-Knowledge (HVZK) [PW22]). We say that ID is perfect special honest-verifier zero-knowledge (perfect HVZK) if there exists a PPT algorithm Sim, a simulator, such that for all $\text{par} \in \text{ISetup}(1^\lambda)$ and all $(pk, sk) \in \text{IGen}(\text{par})$, the following distributions are identical:

$$\{(\text{com}, \text{ch}, \text{resp}) | \text{ch} \leftarrow \text{ChSet}; (\text{com}, \text{ch}, \text{resp}) \leftarrow \text{Tran}(pk, sk, \text{ch})\}$$

and

$$\{(\text{com}, \text{ch}, \text{resp}) | \text{ch} \leftarrow \text{ChSet}; (\text{com}, \text{resp}) \leftarrow \text{Sim}(pk, \text{ch})\}.$$

Kondi and shelat [Ks22] introduced strong special soundness, which is a stronger version of special soundness in the sense that the extractor can extract a secret key of pk from two valid transcripts $(\text{com}, \text{ch}, \text{resp})$ and $(\text{com}, \text{ch}', \text{resp}')$ such that $(\text{ch}, \text{resp}) \neq (\text{ch}', \text{resp}')$. That is, the extractor works even in the case $\text{ch} = \text{ch}'$ and $\text{resp} \neq \text{resp}'$. In this work, we define computational strong special soundness as follows.

Definition 9 (Computational Strong Special Soundness). We say that ID is strong special sound if there exists a PPT algorithm Ext, an extractor, such that for all PPT adversaries $\mathcal{A}$, it holds that

$$\begin{aligned} & \text{Adv}_{\mathcal{A}, \text{ID}}^{\text{SSS}}(\lambda) \\ &:= \Pr \left[\begin{array}{l} (\text{ch}, \text{resp}) \neq (\text{ch}', \text{resp}') \\ \wedge ok = ok' = 1 \\ \wedge \text{VerKey}(\text{par}, pk, sk^*) = 1 \end{array} \middle| \begin{array}{l} \text{par} \leftarrow \text{ISetup}(1^\lambda), \\ (pk, \text{com}, \text{ch}, \text{resp}, \text{ch}', \text{resp}') \leftarrow \mathcal{A}(\text{par}), \\ ok \leftarrow V(pk, \text{com}, \text{ch}, \text{resp}), \\ ok' \leftarrow V(pk, \text{com}, \text{ch}', \text{resp}'), \\ sk^* \leftarrow \text{Ext}(pk, \text{com}, \text{ch}, \text{resp}, \text{ch}', \text{resp}') \end{array} \right] \\ &\leq \text{negl}(\lambda). \end{aligned}$$

Remark 1. Kondi and shelat introduced strong special soundness to relax the requirement, “special soundness plus quasi-unique response” required for the underlying protocol in the original Fischlin transformation [Fis05]. Their original definition does not specify “who” generates two transcripts, does not take into account the failure probability of Ext, and leaves no room for computational assumptions. However, it is clear that special soundness plus quasi-unique response does not imply unconditional strong special soundness. Since they said “Okamoto’s identification protocol satisfies strong special soundness,” they are probably considering the computational one as our definition.

We introduce a variant of key recovery resistance. The following second-key recovery resistance ensures that when ID has multiple secret keys, given a key pair (pk, sk) , it is difficult to find another secret key $sk^* \neq sk$ with respect to pk . For our purpose of constructing a tightly secure signature in the multi-user setting, we define second-key recovery resistance in the multi-user setting.

Definition 10 (Second-Key Recovery in the Multi-User Setting). Let $N = \text{poly}(\lambda)$ be a natural number. We say that ID is second-key recovery resistant in the multi-user setting if for all PPT adversaries $\mathcal{A}$, it holds that

$$\begin{aligned} \text{Adv}_{\mathcal{A}, \text{ID}}^{2^{\text{nd}}\text{KR}}(\lambda) &:= \Pr \left[\begin{array}{c} \text{VerKey}(\text{par}, \text{pk}_{i^*}, \text{sk}^*) = 1 \\ \wedge \text{sk}^* \neq \text{sk}_{i^*} \end{array} \middle| \begin{array}{c} \text{par} \leftarrow \text{ISetup}(1^\lambda), \\ (\text{pk}_i, \text{sk}_i) \leftarrow \text{IGen}(\text{par}) \ \forall i \in [N], \\ (i^*, \text{sk}^*) \leftarrow \mathcal{A}(\text{par}, \{(\text{pk}_i, \text{sk}_i)\}_{i \in [N]}) \end{array} \right] \\ &\leq \text{negl}(\lambda). \end{aligned}$$

3 Signature Scheme via Randomized Fischlin Transformation

In this section, we describe the signature scheme that tightly achieves MU-SUF-CMA-C security. To achieve a tightly secure signature in a multi-user setting with corruption, we should make the security loss independent of the number of users and the number of signing and random-oracle queries. To make the security loss independent of the number of users, we start with an identification scheme that has multiple secret keys per public key, a known technique for achieving tightly secure signatures. Then, to transform the identification into a signature without incurring the security loss from the number of oracle queries, we employ the Fischlin transformation, which allows straight-line extraction.

3.1 Proposed Signature Scheme

Let $\text{ID} = (\text{ISetup}, \text{IGen}, \text{P} = (\text{P}_1, \text{P}_2), \text{V})$ be a canonical identification scheme with the challenge space ChSet . Define the parameters t, ρ, γ, T for the bit-length of the challenges, the number of parallel repetitions, the length of the hash value, and the maximum number of times to retry the signing algorithm, where $\gamma \leq t \leq \lfloor \log |\text{ChSet}| \rfloor$. They must satisfy $\rho \cdot \gamma \geq \lambda$ for security, $2^{t-\gamma} - \log \rho \geq \lambda/T$ for correctness, and $T, \rho = \text{poly}(\lambda)$ and $2^t = \text{poly}(\lambda)$ for a polynomial signing algorithm. Let $\text{H} : \{0, 1\}^* \rightarrow \{0, 1\}^\gamma$ be a hash function modeled as a random oracle. The signature scheme $\text{SIG}[\text{ID}, \text{H}] := (\text{Setup}, \text{KGen}, \text{Sign}^{\text{H}}, \text{Vf}^{\text{H}})$ (in the ROM) is depicted in Figure 2.

We prove that $\text{SIG}[\text{ID}, \text{H}]$ is correct and MU-SUF-CMA-C secure.

Theorem 1. If ID is correct, then the signature scheme $\text{SIG}[\text{ID}, \text{H}]$ is $(1 - \beta)$ -correct for $\beta = 2^{(-2^{t-\gamma} \log e + \log \rho)T}$.

Proof. Randomization of Fischlin transformation does not affect correctness errors, so we can refer to the existing correctness analysis for the original Fischlin transformation. According to [CL24, Section 3], the probability that Sign outputs $\perp$ is at most $(\rho \cdot e^{-2^{t-\gamma}})^T = 2^{(-2^{t-\gamma} \log e + \log \rho)T}$. Since ID is perfectly correct, $\text{SIG}[\text{ID}, \text{H}]$ is $(1 - \beta)$ -correct for $\beta = 2^{(-2^{t-\gamma} \log e + \log \rho)T}$. $\square$

Theorem 2. If ID has κ -bits of commitment min-entropy, K -multiple secret keys for $K \geq 2$, perfect HVZK, strong special sound, and second-key recovery resistant in the multi-user setting, then the signature scheme $\text{SIG}[\text{ID}, \text{H}]$ is MU-SUF-CMA-C secure in the non-programmable random oracle model.

In particular, if there is an adversary $\mathcal{A}$ that breaks the MU-SUF-CMA-C security of $\text{SIG}[\text{ID}, \text{H}]$ in time $T_{\mathcal{A}}$ with success probability $\text{Adv}_{\mathcal{A}, \text{SIG}[\text{ID}, \text{H}]}^{\text{MU-SUF-CMA-C}}(\lambda)$, then there is an algorithm $\mathcal{B}_1$ breaking the strong special soundness of ID in time $T_{\mathcal{B}_1} = \mathcal{O}(T_{\mathcal{A}})$ with probability $\text{Adv}_{\mathcal{B}_1, \text{ID}}^{\text{SSS}}(\lambda)$ and an algorithm $\mathcal{B}_2$ breaking the second-key recovery resistance of ID in the multi-user setting in time $T_{\mathcal{B}_2} = \mathcal{O}(T_{\mathcal{A}})$ with probability $\text{Adv}_{\mathcal{B}_2, \text{ID}}^{2^{\text{nd}}\text{KR}}(\lambda)$ such that

$$\begin{aligned} \text{Adv}_{\mathcal{A}, \text{SIG}[\text{ID}, \text{H}]}^{\text{MU-SUF-CMA-C}}(\lambda) &\leq \text{Adv}_{\mathcal{B}_1, \text{ID}}^{\text{SSS}}(\lambda) + \frac{K}{K-1} \text{Adv}_{\mathcal{B}_2, \text{ID}}^{2^{\text{nd}}\text{KR}}(\lambda) \\ &\quad + \frac{Q_{\text{RO}} + 1}{2^{\rho\kappa}} + \frac{T \cdot Q_{\text{sig}}(Q_{\text{RO}} + T \cdot Q_{\text{sig}})}{2^{\rho\gamma}}. \end{aligned} \tag{1}$$

Here, Q_{RO} and Q_{sig} are the maximum numbers of RO queries and signature queries issued by $\mathcal{A}$, respectively.

Setup(1^λ)	Sign ^H (ssk, m)
1 : $\text{par}' \leftarrow \text{ISetup}(1^\lambda)$ 2 : return $\text{par} := \text{par}'$	1 : foreach $\tau \in [T]$ do 2 : foreach $j \in [\rho]$ do 3 : $(\text{com}_{\tau,j}, \text{st}_{\tau,j}) \leftarrow P_1(\text{pk}, \text{sk})$ 4 : endfor 5 : $\text{com}_\tau := (\text{com}_{\tau,1}, \dots, \text{com}_{\tau,\rho})$ 6 : $\text{pfx}_\tau := (\text{pk}, m, \text{com}_\tau)$ 7 : foreach $j \in [\rho]$ do 8 : $S_j := \emptyset$ 9 : while $S_j \neq \{0, 1\}^t$ do 10 : $\text{ch} \leftarrow_{\$} \{0, 1\}^t \setminus S_j$ 11 : $\text{resp}_{\tau,j,\text{ch}} \leftarrow P_2(\text{ch}, \text{st}_{\tau,j})$ 12 : $h_{\tau,j,\text{ch}} := H(\text{pfx}_\tau, j, \text{ch}, \text{resp}_{\tau,j,\text{ch}})$ 13 : if $h_{\tau,j,\text{ch}} = 0^\gamma$ then 14 : $\text{ch}_{\tau,j} := \text{ch}$ 15 : break // proceed to next j 16 : else 17 : $S_j := S_j \cup \{\text{ch}\}$ 18 : endwhile 19 : endfor 20 : if $\forall j : h_{\tau,j,\text{ch}_{\tau,j}} = 0^\gamma$ then 21 : break // succeed in signing 22 : if $\tau = T$ then 23 : return $\perp$ 24 : endif 25 : $\hat{\tau} := \tau$ 26 : $\sigma := (\text{com}_{\hat{\tau},j}, \text{ch}_{\hat{\tau},j}, \text{resp}_{\hat{\tau},j,\text{ch}_{\hat{\tau},j}})_{j \in [\rho]}$ 27 : return σ
KGen (par)	
1 : $(\text{pk}, \text{sk}) \leftarrow \text{IGen}(\text{par})$ 2 : return $(\text{svk}, \text{ssk}) := (\text{pk}, (\text{pk}, \text{sk}))$	
Vf^H (svk, m , σ)	
1 : $(\text{com}_j, \text{ch}_j, \text{resp}_j)_{j \in [\rho]} := \sigma$ 2 : $\text{com} := (\text{com}_1, \dots, \text{com}_\rho)$ 3 : $\text{pfx} := (\text{pk}, m, \text{com})$ 4 : foreach $j \in [\rho]$ do 5 : if $H(\text{pfx}, j, \text{ch}_j, \text{resp}_j) \neq 0^\gamma$ then 6 : return 0 7 : if $V(\text{pk}, \text{com}_j, \text{ch}_j, \text{resp}_j) = 0$ then 8 : return 0 9 : endfor 10 : return 1	

Fig. 2: The signature scheme SIG[ID, H].

This theorem shows that the MU-SUF-CMA-C security of our signature scheme is *tightly reduced* to the security against the second-key recovery resistance and the strong special soundness of ID in the non-programmable ROM. Here, we provide a proof sketch. The full proof is provided in Appendix A.

Proof (sketch). For simplicity, assume that the underlying ID scheme has perfect strong special soundness ($\text{Adv}_{\mathcal{A}, \text{SIG}[\text{ID}, \text{H}]}^{\text{MU-SUF-CMA-C}}(\lambda) = 0$), honestly generated com is always fresh (i.e., $1/2^{\rho\kappa} \approx 0$), and the adversary cannot find ch_j that yields the hash value 0^γ for each $j \in [\rho]$ with one oracle query (i.e., $1/2^{\rho\gamma} \approx 0$).

Consider the following reduction $\mathcal{R}$ that reduces the MU-SUF-CMA-C security of SIG[ID, H] to the second-key recovery resistance. On input $(\text{pk}_i, \text{sk}_i)_{i \in [N]}$, $\mathcal{R}$ runs $\mathcal{A}$ with input $(\text{pk}_i)_{i \in [N]}$. $\mathcal{R}$ simulates the signing and the corruption oracles using sk_i . On the simulation of the RO, $\mathcal{R}$ always returns a random hash value (without programming), and maintains the query-answer list L as usual. In addition, for each query $(\text{pk}_i, m, \text{com}, j, \text{ch}, \text{resp})$, $\mathcal{R}$ checks if $(\text{com}_j, \text{ch}, \text{resp})$ is a valid transcript w.r.t. pk_i , i.e., $V(\text{pk}_i, \text{com}_j, \text{ch}, \text{resp}) = 1$ holds, and if there is another valid query $(\text{pk}_i, m, \text{com}, j, \text{ch}', \text{resp}')$ that shares the same $(\text{pk}_i, m, \text{com}, j)$ but $(\text{ch}', \text{resp}') \neq (\text{ch}, \text{resp})$ in L . If exists, $\mathcal{R}$ sets the flag F_{find} and computes $\hat{\text{sk}} = \text{Ext}(\text{pk}_i, \text{com}_j, \text{ch}, \text{resp}, \text{ch}')$,

resp'). The perfect strong special soundness ensures that $\hat{\text{sk}}$ is a valid secret key w.r.t. pk_i , and $\mathcal{R}$ successfully finds the second secret key if $\hat{\text{sk}} \neq \text{sk}_i$.

Next, we estimate the probability that $\hat{\text{sk}} = \text{sk}_i$ holds. If randomized Fischlin transformation preserves the perfect (HV)ZK property of ID, we can conclude that $\hat{\text{sk}} = \text{sk}_i$ holds with probability $1/K$, that implies $\mathcal{R}$'s success probability is $(K-1)/K$ times that of $\mathcal{A}$. However, the transformation does not preserve the perfect (HV)ZK property [Sch22]. More precisely, simulated signatures themselves never leak information about the used secret key sk_i , but the responses of the RO for special queries depend on sk_i . Let $(\text{com}, \text{ch}, \text{resp})$ be a valid transcript w.r.t. pk_i included in a signature generated by $\mathcal{R}$. During the signing process, $\mathcal{R}$ internally issues not only $(\text{pk}_i, m, \text{com}, j, \text{ch}, \text{resp})$ but also other queries $(\text{pk}_i, m, \text{com}, j, \text{ch}'(\neq \text{ch}), \text{resp}')$ to the RO, and the conditional probability distribution of $H(\text{pk}_i, m, \text{com}, j, \text{ch}', \text{resp}')$ under the condition “ $(\text{com}_j, \text{ch}', \text{resp}')$ was not included in the signature” is different from the uniform distribution. Furthermore, the set of such $(\text{pk}_i, m, \text{com}, j, \text{ch}', \text{resp}')$ that has a biased probability distribution depends on the used secret key. Therefore, $\mathcal{A}$ may get some information about sk_i by asking to the RO for such $(\text{pk}_i, m, \text{com}, j, \text{ch}', \text{resp}')$.

We avoid this situation using the observation that once $(\text{pk}_i, m, \text{com}, j, \text{ch}', \text{resp}')$ that may leak the information about sk_i is queried to the RO, the flag F_{find} is set. In other words, $\mathcal{A}$'s view up to the point of issuing a query that triggers the flag to be set is independent of sk_i . Thus, we can conclude that $\hat{\text{sk}}$ extracted from the trigger query is equal to sk_i with probability $1/K$. $\square$

Remark 2. Although Kondi-shelat claimed that an NIZK obtained by randomized Fischlin transform is statistically ZK, Scheurer [Sch22] pointed out that it is not even computationally ZK. Recently, Klooß et al. [KRW24] proved that it is computationally ZK if an exponentially large challenge space is used in the transformation in the PROM.

In contrast, a polynomial-size challenge space is enough in our construction. This is because a weaker notion of ZK is sufficient to guarantee the security of our signature scheme. More precisely, our construction requires ZK property against a distinguisher who is not allowed to choose statement-witness pairs but is given an honestly generated statement. In fact, such a weaker notion is satisfied by the original Fischlin transformation [Fis05], and by slightly modifying the proof of Theorem 2, we can show the security of our construction that uses the original Fischlin transformation.

Furthermore, the tight security of our construction is proven in the non-programmable ROM, unlike the proof of Klooß et al. This can be achieved by our observation that NIZKs obtained by the transformation satisfy *unconditional* ZK property until a query is requested to the RO that triggers the flag F_{find} to be set. Since the ZK property is used in our proof to show the fact that no information about sk_i is leaked to $\mathcal{A}$ before requesting such a query, we can enjoy the unconditional ZK in the NPROM.

Remark 3. Yoshioka et al. [YOH24] showed an impossibility result stating that a signature scheme cannot achieve tight security if it satisfies a specific set of conditions. The above construction from the Fischlin transformation circumvents the impossibility by bypassing the property “random oracle statistically close”, which is one of these conditions.

3.2 Improving Efficiency

If the verifier algorithm of the underlying ID scheme can be represented as

$$\text{V}(\text{pk}, \text{com}, \text{ch}, \text{resp}) = 1 \iff \text{com} = f^{\text{V}}(\text{pk}, \text{ch}, \text{resp}) \quad (2)$$

for some efficiently computable function f^{V} , we can eliminate com in the signature as in Schnorr signature. In this case, $\text{Vf}^{\text{H}}(\text{svk} = \text{pk}, m, \sigma = (\text{ch}_j, \text{resp}_j)_j)$ first reconstructs $\text{com}_j := f^{\text{V}}(\text{pk}, \text{ch}_j, \text{resp}_j)$, $\text{com} := (\text{com}_j)_j$, and then checks if $H(\text{pk}, m, \text{com}, j, \text{ch}_j, \text{resp}_j) = 0^\gamma$ holds for all $j \in [\rho]$.

4 Instantiations of Signature Schemes

In this section, we provide concrete instantiations of tightly MU-SUF-CMA-C-secure signatures from our framework. We consider instantiations based on classical assumptions (DL, RSA, and factoring) via the

$\text{ISetup}_{\text{Oka}}(1^\lambda)$	$\text{POka,1}(\text{pk}, \text{sk})$
1 : $(\mathbb{G}, q, g) \leftarrow \text{GGen}(1^\lambda)$	1 : $(r_1, r_2) \leftarrow \$_{(\mathbb{Z}_q)^2}$
2 : $\alpha \leftarrow \$_{\mathbb{Z}_q}; g_1 := g^\alpha$	2 : $R := g^{r_1} g_1^{r_2}$
3 : $\text{ChSet} := \mathbb{Z}_q$	3 : $\text{com} := R$
4 : return $\text{par} := (\mathbb{G}, q, g, g_1)$	4 : $\text{st} := (\text{sk}, r_1, r_2)$
	5 : return (com, st)
$\text{IGen}_{\text{Oka}}(\text{par})$	$\text{POka,2}(\text{st}, \text{ch})$
1 : $\text{sk} := (s_1, s_2) \leftarrow \$_{(\mathbb{Z}_q)^2}$	1 : $y_1 := r_1 + \text{ch} \cdot s_1 \bmod q$
2 : $\text{pk} := g^{s_1} g_1^{s_2}$	2 : $y_2 := r_2 + \text{ch} \cdot s_2 \bmod q$
3 : return (pk, sk)	3 : $\text{resp} := (y_1, y_2)$
$\text{VOka}(\text{par}, \text{pk}, \text{com}, \text{ch}, \text{resp})$	4 : return resp
1 : $(y_1, y_2) := \text{resp}$	
2 : if $\text{com} = g^{y_1} g_1^{y_2} / \text{pk}^{\text{ch}}$ then	
3 : return 1	
4 : else return 0	

Fig. 3: The Okamoto identification scheme ID_{Oka} .

Okamoto identification [Oka93] and its variants and an instantiation from isogenies based on Couveignes-Stolbunov identification [Cou06,BKV19,Sto12] with the Parallel-OR technique [CDS94].

4.1 Instantiation Based on Classical Assumptions

Okamoto identification scheme based on DL. Okamoto scheme [Oka93, Scheme 1], which is based on the discrete-logarithm (DL) assumption, is one of the most important instantiations of our framework. Let GGen be a PPT algorithm that on input 1^λ generates a prime q , a multiplicative group $\mathbb{G}$ with order q , and a generator $g \in \mathbb{G}$, and outputs $(\mathbb{G}, q, g)$. Okamoto scheme $\text{ID}_{\text{Oka}} := (\text{ISetup}_{\text{Oka}}, \text{IGen}_{\text{Oka}}, \text{POka}, \text{VOka})$ is defined in Figure 3.

ID_{Oka} has correctness, multiple secret keys, perfect HVZK, strong special soundness, and second-key recovery resistance in the multi-user setting. Before proving them, we introduce the DL assumption.

Definition 11 (Discrete Logarithm (DL) Assumption). *We say that DL assumption holds for GGen if for all PPT adversaries $\mathcal{A}$, it holds that*

$$\text{Adv}_{\mathcal{A}, \text{GGen}}^{\text{DL}}(\lambda) := \Pr \left[g^{\alpha'} = g_1 \mid \begin{array}{l} (\mathbb{G}, q, g) \leftarrow \text{GGen}(1^\lambda), \\ \alpha \leftarrow \$_{\mathbb{Z}_q}, \\ g_1 := g^\alpha, \\ \alpha' \leftarrow \mathcal{A}(\mathbb{G}, q, g, g_1) \end{array} \right] \leq \text{negl}(\lambda).$$

We now prove the properties of ID_{Oka} .

Theorem 3. ID_{Oka} is correct, and it has $\log q$ -bits of commitment min-entropy and q -multiple secret keys.

Proof. The correctness of ID_{Oka} is clear (proved in [Oka93]). The commitment of ID_{Oka} consists of a randomly chosen group element over $\mathbb{G}$ with order q . Thus, ID_{Oka} has $\log q$ -bits of commitment min-entropy. Finally, for each pk , there are q pairs of (s_1, s_2) such that $\text{pk} = g^{s_1} g_1^{s_2}$. Thus, ID_{Oka} has q -multiple secret keys. $\square$

Theorem 4. Under the DL assumption, ID_{Oka} is strong special sound. More precisely, there exists an extractor Ext such that, for any adversary $\mathcal{A}$ breaking strong special soundness of ID_{Oka} with advantage $\text{Adv}_{\mathcal{A}, \text{ID}_{\text{Oka}}}^{\text{SSS}}(\lambda)$, there exists a DL solver $\mathcal{B}$ whose advantage is

$$\text{Adv}_{\mathcal{B}, \text{GGen}}^{\text{DL}}(\lambda) = \text{Adv}_{\mathcal{A}, \text{ID}_{\text{Oka}}}^{\text{SSS}}(\lambda).$$

This means that $\text{Adv}_{\mathcal{A}, \text{ID}_{\text{Oka}}}^{\text{SSS}}(\lambda)$ is upper bounded by $\max_{\mathcal{B}} \text{Adv}_{\mathcal{B}, \text{GGen}}^{\text{DL}}(\lambda)$.

Proof. We define the extractor Ext as follows. Let $(\text{pk}, \text{com}, \text{ch}, \text{resp}, \text{ch}', \text{resp}')$ be Ext 's input. If $\text{ch} \neq \text{ch}'$, Ext computes

$$\begin{aligned} s_1^* &:= (y_1 - y'_1)/(\text{ch} - \text{ch}') \bmod q, \\ s_2^* &:= (y_2 - y'_2)/(\text{ch} - \text{ch}') \bmod q \end{aligned}$$

and outputs $\text{sk}^* := (s_1^*, s_2^*)$. Otherwise, Ext outputs $\perp$.

When $\text{ch} \neq \text{ch}'$, it is easy to see that Ext outputs a valid secret key. So, $\text{Adv}_{\mathcal{A}, \text{ID}_{\text{Oka}}}^{\text{SSS}}(\lambda)$ is the probability that $\mathcal{A}$ outputs $(\text{pk}, \text{com}, \text{ch}, \text{resp}, \text{ch}', \text{resp}')$ such that $\text{ch} = \text{ch}'$, $\text{resp} \neq \text{resp}'$, and $V(\text{pk}, \text{com}, \text{ch}, \text{resp}) = V(\text{pk}, \text{com}, \text{ch}', \text{resp}') = 1$. Note that $(\text{ch} = \text{ch}' \wedge \text{resp} \neq \text{resp}')$ implies $y_2 \neq y'_2$ if $V(\text{pk}, \text{com}, \text{ch}, (y_1, y_2)) = V(\text{pk}, \text{com}, \text{ch}', (y'_1, y'_2)) = 1$.

Now we construct a PPT algorithm $\mathcal{B}$ that solves the DL problem by using $\mathcal{A}$. Upon receiving a DL instance $(\mathbb{G}, q, g, g_1)$, $\mathcal{B}$ executes $\mathcal{A}$ on input $\text{par} := (\mathbb{G}, q, g, g_1)$. If $\mathcal{A}$ outputs pk and two valid transcripts $(\text{com}, \text{ch}, \text{resp})$ and $(\text{com}, \text{ch}', \text{resp}')$ with respect to pk such that $\text{ch} = \text{ch}'$ and $y_2 \neq y'_2$, $\mathcal{B}$ can compute a DL of g_1 as

$$\alpha := (y_1 - y'_1)/(y'_2 - y_2).$$

Clearly, $\text{Adv}_{\mathcal{B}, \text{GGen}}^{\text{DL}}(\lambda) = \text{Adv}_{\mathcal{A}, \text{ID}_{\text{Oka}}}^{\text{SSS}}(\lambda)$ holds. In addition, the running time of $\mathcal{B}$ is $\mathcal{A}$'s running time plus $\text{poly}(\lambda)$. $\square$

Theorem 5. ID_{Oka} is perfect HVZK.

Proof. Consider the following simulator: On input (pk, ch) , choose $y_1, y_2 \leftarrow \mathbb{Z}_q$, compute $R := g^{y_1} g_1^{y_2} / \text{pk}^{\text{ch}}$, and output $(R, (y_1, y_2))$.

It is easy to confirm that the simulator's output has the same distribution as the real transcript between an honest prover and an honest verifier. $\square$

Theorem 6. Under the DL assumption, ID_{Oka} satisfies the second-key recovery resistance. In particular, if there is an adversary $\mathcal{A}$ that breaks the second-key recovery resistance in time $T_{\mathcal{A}}$ with success probability $\text{Adv}_{\mathcal{A}, \text{ID}}^{2^{\text{ndKR}}}(\lambda)$, then there is an algorithm $\mathcal{B}$ solving the DL problem in time $T_{\mathcal{B}} = T_{\mathcal{A}} + N \cdot \text{poly}(\lambda)$ with probability $\text{Adv}_{\mathcal{B}, \text{GGen}}^{\text{DL}}(\lambda) = \text{Adv}_{\mathcal{A}, \text{ID}}^{2^{\text{ndKR}}}(\lambda)$.

Proof. Let $\mathcal{A}$ be an adversary that breaks the second-key recovery resistance. Consider the following DL solver $\mathcal{B}$ that uses $\mathcal{A}$ as a subroutine: Upon receiving a DL instance $(\mathbb{G}, q, g, g_1)$, $\mathcal{B}$ sets $\text{par} := (\mathbb{G}, q, g, g_1)$ and generates $(\text{pk}_i, \text{sk}_i) \leftarrow \text{IGen}(\text{par})$ for each $i \in [N]$. $\mathcal{B}$ executes $\mathcal{A}$ on input $(\text{par}, \{(\text{pk}_i, \text{sk}_i)\}_{i \in [N]})$ and receives (i^*, sk^*) from $\mathcal{A}$ such that $\text{sk}^* = (s_1^*, s_2^*) \neq \text{sk}_{i^*} = (s_{1,i^*}, s_{2,i^*})$ and $g^{s_1^*} g_1^{s_2^*} = \text{pk}_{i^*} = g^{s_{1,i^*}} g_1^{s_{2,i^*}}$. Then, $\mathcal{B}$ computes

$$\alpha := (s_1^* - s_{1,i^*}) / (s_{2,i^*} - s_2^*) \bmod q,$$

and outputs α as the solution of the DL instance.

We can verify that if $\mathcal{A}$ breaks the second-key recovery resistance, $\mathcal{B}$ solves the given DL instance. Therefore, we have $\text{Adv}_{\mathcal{B}, \text{GGen}}^{\text{DL}}(\lambda) = \text{Adv}_{\mathcal{A}, \text{ID}}^{2^{\text{ndKR}}}(\lambda)$. Note that the running time of $\mathcal{B}$ is $T_{\mathcal{B}} = T_{\mathcal{A}} + N \cdot \text{poly}(\lambda)$ since $\mathcal{B}$ executes $\mathcal{A}$ once and prepares N key pairs. $\square$

By instantiating $\text{SIG}[\text{ID}, \text{H}]$ with ID_{Oka} , we obtain a signature scheme, $\text{SIG}[\text{ID}_{\text{Oka}}, \text{H}]$, whose MU-SUF-CMA-C security is tightly implied from the DL assumption.

Corollary 1. *Under the DL assumption, $\text{SIG}[\text{ID}_{\text{Oka}}, \text{H}]$ is MU-SUF-CMA-C secure in the non-programmable random oracle model. In particular, for any adversary $\mathcal{A}$ that breaks the MU-SUF-CMA-C security of $\text{SIG}[\text{ID}_{\text{Oka}}, \text{H}]$ in time $T_{\mathcal{A}}$ with probability $\text{Adv}_{\mathcal{A}, \text{SIG}[\text{ID}_{\text{Oka}}, \text{H}]}^{\text{MU-SUF-CMA-C}}(\lambda)$, there is an algorithm $\mathcal{B}$ solving the DL problem in time $T_{\mathcal{B}} = \mathcal{O}(T_{\mathcal{A}})$ with probability $\text{Adv}_{\mathcal{B}, \text{Gen}}^{\text{DL}}(\lambda)$ such that*

$$\text{Adv}_{\mathcal{A}, \text{SIG}[\text{ID}_{\text{Oka}}, \text{H}]}^{\text{MU-SUF-CMA-C}}(\lambda) \leq 3\text{Adv}_{\mathcal{B}, \text{Gen}}^{\text{DL}}(\lambda) + \frac{Q_{\text{RO}} + 1}{2^{\rho \log_2 q}} + \frac{T \cdot Q_{\text{sig}}(Q_{\text{RO}} + T \cdot Q_{\text{sig}})}{2^{\rho \gamma}}.$$

Okamoto identification scheme based on RSA or Factoring. In exactly the same way, tightly secure signature schemes based on the RSA assumption and the factoring assumption are realized by applying our generic construction to the variants of the Okamoto scheme proposed in [Oka93, Scheme 2] and [Fis02], respectively.

Corollary 2. *Under the RSA assumption, a tight MU-SUF-CMA-C secure signature exists in the non-programmable random oracle model.*

Corollary 3. *Under the factoring assumption, a tight MU-SUF-CMA-C secure signature exists in the non-programmable random oracle model.*

4.2 Instantiation from Group Action

We will show a group action-based tightly MU-SUF-CMA-C-secure signature derived from our framework. We first show that the so-called Parallel-OR identification scheme [CDS94] meets the requirements of our framework. Then, we provide a concrete instantiation of the Parallel-OR identification scheme based on group action.

Parallel-OR identification scheme. Let $\text{ID} = (\text{ISetup}, \text{IGen}, \text{P}, \text{V})$ be a canonical identification scheme with ℓ bits challenges and let Sim be a special honest verifier zero-knowledge simulator of ID . Then the Parallel-OR identification scheme $\text{ID}_{\text{OR}}[\text{ID}] := (\text{ISetup}_{\text{OR}}, \text{IGen}_{\text{OR}}, \text{P}_{\text{OR}}, \text{V}_{\text{OR}})$ is defined as shown in Figure 4.

For completeness, we show that $\text{ID}_{\text{OR}}[\text{ID}]$ has all properties required for the underlying identification scheme to apply our framework in Section 3, if ID has correctness, single secret key, perfect HVZK, strong special soundness, and the following basic properties.

Definition 12 (Key Recovery [KMP16, Definition 2.3]). *We say that ID is key recovery resistant if for all PPT adversaries $\mathcal{A}$, it holds that*

$$\text{Adv}_{\mathcal{A}, \text{ID}}^{\text{KR}}(\lambda) := \Pr \left[\text{VerKey}(\text{par}, \text{pk}, \text{sk}^*) = 1 \mid \begin{array}{l} \text{par} \leftarrow \text{ISetup}(1^\lambda), \\ (\text{pk}, \text{sk}) \leftarrow \text{IGen}(\text{par}), \\ \text{sk}^* \leftarrow \mathcal{A}(\text{par}, \text{pk}) \end{array} \right] \leq \text{negl}(\lambda).$$

Definition 13 (Random Self-Reducibility [KMP16, Definition 2.5]). *We say that ID is random self-reducible if there is a PPT algorithm ReRand and a deterministic algorithm DeRand such that, for all $(\text{pk}, \text{sk}) \in \text{IGen}(\text{par})$:*

- pk' and pk'' have the same distribution, where $(\text{pk}', \text{td}') \leftarrow \text{ReRand}(\text{par}, \text{pk})$ is the rerandomized public key and $(\text{pk}'', \text{sk}'') \leftarrow \text{IGen}(\text{par})$ is a freshly-generated key pair.
- For all $(\text{pk}', \text{td}') \in \text{ReRand}(\text{par}, \text{pk})$, for all $(\text{pk}', \text{sk}') \in \text{IGen}(\text{par})$, and $\text{sk}^* \leftarrow \text{DeRand}(\text{pk}, \text{pk}', \text{sk}', \text{td}')$, we have $\text{VerKey}(\text{par}, \text{pk}, \text{sk}^*) = 1$. That is, DeRand returns a valid secret key sk^* with respect to pk , given any valid secret key sk' for pk' .

We now show the properties of $\text{ID}_{\text{OR}}[\text{ID}]$.

$\text{IGen}_{\text{OR}}(\text{par})$	$\text{P}_{\text{OR},1}(\text{pk} = (\text{pk}_0, \text{pk}_1), \text{sk} = (b, \text{sk}_b))$
1 : $b \leftarrow_{\$} \{0, 1\}$	1 : $\text{ch}_{1-b} \leftarrow_{\$} \{0, 1\}^\ell$
2 : $(\text{pk}_0, \text{sk}_0) \leftarrow \text{IGen}(\text{par})$	2 : $(\text{com}_{1-b}, \text{resp}_{1-b}) \leftarrow \text{Sim}(\text{pk}_{1-b}, \text{ch}_{1-b})$
3 : $(\text{pk}_1, \text{sk}_1) \leftarrow \text{IGen}(\text{par})$	3 : $(\text{com}_b, \text{st}_b) \leftarrow \text{P}_1(\text{pk}_b, \text{sk}_b)$
4 : $\text{pk} := (\text{pk}_0, \text{pk}_1); \text{sk} := (b, \text{sk}_b)$	4 : $\text{com} := (\text{com}_0, \text{com}_1)$
5 : return (pk, sk)	5 : $\text{st} := (b, \text{st}_b, \text{ch}_{1-b}, \text{resp}_{1-b})$
	6 : return (com, st)
$\text{V}_{\text{OR}}(\text{pk}, \text{com}, \text{ch}, \text{resp})$	$\text{P}_{\text{OR},2}(\text{st}, \text{ch})$
1 : $\text{ch}_1 := \text{ch} \oplus \text{ch}_0$	1 : $\text{ch}_b := \text{ch} \oplus \text{ch}_{1-b}$
2 : $b_0 \leftarrow \text{V}(\text{pk}_0, \text{com}_0, \text{ch}_0, \text{resp}_0)$	2 : $\text{resp}_b \leftarrow \text{P}_2(\text{ch}_b, \text{st}_b)$
3 : $b_1 \leftarrow \text{V}(\text{pk}_1, \text{com}_1, \text{ch}_1, \text{resp}_1)$	3 : $\text{resp} := (\text{ch}_0, \text{resp}_0, \text{resp}_1)$
4 : if $b_0 = 1 \wedge b_1 = 1$ then	4 : return resp
5 : return 1	
6 : else return 0	

Fig. 4: The Parallel-OR identification scheme $\text{ID}_{\text{OR}}[\text{ID}]$ constructed from an identification scheme ID , where $\text{ISetup}_{\text{OR}} := \text{ISetup}$.

Theorem 7. *If ID has correctness and κ -bits of commitment min-entropy, then $\text{ID}_{\text{OR}}[\text{ID}]$ also has correctness and 2κ -bits of commitment min-entropy. If ID has a single secret key, $\text{ID}_{\text{OR}}[\text{ID}]$ has 2-multiple secret keys.*

Proof. The correctness of $\text{ID}_{\text{OR}}[\text{ID}]$ is clear. The commitment of $\text{ID}_{\text{OR}}[\text{ID}]$ consists of two independent commitments of ID . Thus, $\text{ID}_{\text{OR}}[\text{ID}]$ has 2κ -bits of commitment min-entropy. Finally, if the underlying identification scheme ID has a single secret key per public key, there are two secret keys $(0, \text{sk}_0)$ and $(1, \text{sk}_1)$ for each pk of $\text{ID}_{\text{OR}}[\text{ID}]$. Thus, $\text{ID}_{\text{OR}}[\text{ID}]$ has 2-multiple secret keys. $\square$

Theorem 8. *If ID is strong special sound, $\text{ID}_{\text{OR}}[\text{ID}]$ is also strong special sound. More precisely, if there exists an extractor Ext_{ID} such that, for any adversary $\mathcal{A}$, its advantage is at most $\text{Adv}_{\text{ID}}^{\text{SSS}}(\lambda)$, then there exists an extractor $\text{Ext}_{\text{ID}_{\text{OR}}[\text{ID}]}$ such that, for any adversary $\mathcal{B}$, its advantage is at most $\text{Adv}_{\text{ID}_{\text{OR}}[\text{ID}]}^{\text{SSS}}(\lambda) = \text{Adv}_{\text{ID}}^{\text{SSS}}(\lambda)$.*

Proof. Let Ext_{ID} be an extractor of ID . Consider the extractor $\text{Ext}_{\text{ID}_{\text{OR}}[\text{ID}]}$ of $\text{ID}_{\text{OR}}[\text{ID}]$ as follows. Given two valid transcripts $((\text{com}_0, \text{com}_1), \text{ch}, (\text{ch}_0, \text{resp}_0, \text{resp}_1))$ and $((\text{com}_0, \text{com}_1), \text{ch}', (\text{ch}'_0, \text{resp}'_0, \text{resp}'_1))$ with respect to $\text{pk} = (\text{pk}_0, \text{pk}_1)$,

- if $\text{ch}_0 \neq \text{ch}'_0$ or $\text{resp}_0 \neq \text{resp}'_0$, $\text{Ext}_{\text{ID}_{\text{OR}}[\text{ID}]}$ outputs

$$(0, \text{Ext}_{\text{ID}}(\text{pk}_0, \text{com}_0, \text{ch}_0, \text{resp}_0, \text{ch}'_0, \text{resp}'_0)).$$

- Otherwise, $\text{ch}_1 \neq \text{ch}'_1$ or $\text{resp}_1 \neq \text{resp}'_1$ must hold, where $\text{ch}_1 := \text{ch} \oplus \text{ch}_0, \text{ch}'_1 := \text{ch}' \oplus \text{ch}'_0$. In this case, $\text{Ext}_{\text{ID}_{\text{OR}}[\text{ID}]}$ outputs

$$(1, \text{Ext}_{\text{ID}}(\text{pk}_1, \text{com}_1, \text{ch}_1, \text{resp}_1, \text{ch}'_1, \text{resp}'_1)).$$

Let $\mathcal{B}$ be an arbitrary algorithm that breaks strong special soundness of $\text{ID}_{\text{OR}}[\text{ID}]$. In order to estimate $\mathcal{B}$'s advantage, consider $\mathcal{A}_{\mathcal{B}}$ that works as follows: On input par , $\mathcal{A}_{\mathcal{B}}$ runs $\mathcal{B}$ and obtains $\mathcal{B}$'s output $(\text{pk}_0, \text{pk}_1), (\text{com}_0, \text{com}_1), \text{ch}, (\text{ch}_0, \text{resp}_0, \text{resp}_1), \text{ch}', (\text{ch}'_0, \text{resp}'_0, \text{resp}'_1)$. If $\text{ch}_0 \neq \text{ch}'_0$ or $\text{resp}_0 \neq \text{resp}'_0$, $\mathcal{A}_{\mathcal{B}}$ outputs $(\text{pk}_0, \text{com}_0, \text{ch}_0, \text{resp}_0, \text{ch}'_0, \text{resp}'_0)$, otherwise outputs $(\text{pk}_1, \text{com}_1, \text{ch}_1, \text{resp}_1, \text{ch}'_1, \text{resp}'_1)$, where $\text{ch}_1 := \text{ch} \oplus \text{ch}_0, \text{ch}'_1 := \text{ch}' \oplus \text{ch}'_0$.

From the assumption, for this $\mathcal{A}_{\mathcal{B}}$, Ext_{ID} successfully extracts a valid secret key with probability at least $1 - \text{Adv}_{\text{ID}}^{\text{SSS}}(\lambda)$. On the other hand, the distribution of Ext_{ID} 's input come from $\mathcal{A}_{\mathcal{B}}$ is identical to the distribution of Ext_{ID} 's input when Ext_{ID} is used as a subroutine of $\text{Ext}_{\text{ID}_{\text{OR}}[\text{ID}]}$. Therefore, $\text{Ext}_{\text{ID}_{\text{OR}}[\text{ID}]}$ obtains a valid sk_0 or sk_1 with probability at least $1 - \text{Adv}_{\text{ID}}^{\text{SSS}}(\lambda)$. That is, $\mathcal{B}$'s advantage is at most $\text{Adv}_{\text{ID}}^{\text{SSS}}(\lambda)$. $\square$

Theorem 9. *If ID is perfect HVZK, ID_{OR}[ID] is also perfect HVZK.*

Proof. Let Sim be a simulator for ID. We can easily construct a simulator Sim_{OR} for ID_{OR}[ID] by using Sim as follows: Sim_{OR}'s input is $\mathbf{pk} = (\mathbf{pk}_0, \mathbf{pk}_1)$ and $\mathbf{ch} \in \{0, 1\}^\ell$. First, choose $\mathbf{ch}_0$ randomly from $\{0, 1\}^\ell$ and set $\mathbf{ch}_1 := \mathbf{ch} \oplus \mathbf{ch}_0$. Run $(\mathbf{com}_b, \mathbf{resp}_b) \leftarrow \text{Sim}(\mathbf{pk}_b, \mathbf{ch}_b)$ for each $b \in \{0, 1\}$, and output $\mathbf{com} := (\mathbf{com}_0, \mathbf{com}_1), \mathbf{resp} := (\mathbf{ch}_0, \mathbf{resp}_0, \mathbf{resp}_1)$. It is easy to see Sim_{OR}'s simulation is perfect from the fact that ID is perfect HVZK. $\square$

Theorem 10. *If ID is key recovery resistant, random self-reducible, and has a single secret key, then ID_{OR}[ID] is the second-key recovery resistant in the multi-user setting. In particular, if there is an adversary $\mathcal{A}$ that breaks the second-key recovery resistance in time $T_{\mathcal{A}}$ with success probability $\text{Adv}_{\mathcal{A}, \text{ID}_{\text{OR}}[\text{ID}]}^{2^{\text{nd}}\text{KR}}(\lambda)$, then there is an algorithm $\mathcal{B}$ breaking the key recovery resistance of ID in time $T_{\mathcal{B}} = T_{\mathcal{A}} + N \cdot \text{poly}(\lambda)$ with probability $\text{Adv}_{\mathcal{B}, \text{ID}}^{\text{KR}}(\lambda) = \text{Adv}_{\mathcal{A}, \text{ID}_{\text{OR}}[\text{ID}]}^{2^{\text{nd}}\text{KR}}(\lambda)$.*

Proof. Let $\mathcal{A}$ be an adversary that breaks the second-key recovery resistance of ID_{OR}[ID]. We show a reduction $\mathcal{B}$ that breaks the key recovery resistance of ID by using $\mathcal{A}$. The description of $\mathcal{B}$ is as follows.

Upon receiving a parameter par and a public key $\mathbf{pk}$, $\mathcal{B}$ generates $(\mathbf{pk}_i, \mathbf{sk}_i)$ for each $i \in [N]$ as follows:

1. Sample $b_i \leftarrow \{0, 1\}$.
2. Generate $(\mathbf{pk}_{i, b_i}, \mathbf{sk}_{i, b_i}) \leftarrow \text{IGen}(\text{par})$.
3. Generate $(\mathbf{pk}_{i, 1-b_i}, \mathbf{td}_i) \leftarrow \text{ReRand}(\text{par}, \mathbf{pk})$.
4. Set $\mathbf{pk}_i := (\mathbf{pk}_{i, 0}, \mathbf{pk}_{i, 1})$ and $\mathbf{sk}_i := (b_i, \mathbf{sk}_{i, b_i})$.

Then, $\mathcal{B}$ executes $\mathcal{A}$ on input $(\text{par}, \{(\mathbf{pk}_i, \mathbf{sk}_i)\}_{i \in [N]})$ and receives $(i^*, (b', \mathbf{sk}'_{i^*, b'}))$ from $\mathcal{A}$. Then, $\mathcal{B}$ computes $\mathbf{sk}^* \leftarrow \text{DeRand}(\mathbf{pk}, \mathbf{pk}_{i^*, 1-b_{i^*}}, \mathbf{sk}'_{i^*, b'}, \mathbf{td}_{i^*})$ and outputs $\mathbf{sk}^*$.

$\mathcal{B}$ perfectly simulates the second-key recovery resistance game, since the random self-reducibility ensures that the randomized public key embedded in each $\mathbf{pk}_{i, 1-b_i}$ is distributed identically to a fresh public key. Moreover, if $\mathcal{A}$ breaks the second-key recovery resistance, its output $(i^*, (b', \mathbf{sk}'_{i^*, b'}))$ must satisfy $b' = 1 - b_{i^*}$ and $\text{VerKey}(\text{par}, \mathbf{pk}_{i^*, b'}, \mathbf{sk}'_{i^*, b'}) = 1$ since ID_{OR}[ID] has exactly two valid secret keys. Therefore, $\mathcal{B}$ extracts the secret key of given $\mathbf{pk}$, and we have $\text{Adv}_{\mathcal{B}, \text{ID}}^{\text{KR}}(\lambda) = \text{Adv}_{\mathcal{A}, \text{ID}_{\text{OR}}[\text{ID}]}^{2^{\text{nd}}\text{KR}}(\lambda)$. The running time of $\mathcal{B}$ is $T_{\mathcal{B}} = T_{\mathcal{A}} + N \cdot \text{poly}(\lambda)$ since $\mathcal{B}$ executes $\mathcal{A}$ once and prepares N key pairs. $\square$

Remark 4. Let ID be an identification scheme whose verifier algorithm is represented as in Eq.(2) using a function f^V , and let define the function f_{OR}^V as

$$f_{\text{OR}}^V(\mathbf{pk}_{\text{OR}}, \mathbf{ch}_{\text{OR}}, \mathbf{resp}_{\text{OR}}) := (f^V(\mathbf{pk}_0, \mathbf{ch}_0, \mathbf{resp}_0), f^V(\mathbf{pk}_1, \mathbf{ch}_{\text{OR}} \oplus \mathbf{ch}_0, \mathbf{resp}_1)),$$

where $\mathbf{pk}_{\text{OR}} = (\mathbf{pk}_0, \mathbf{pk}_1), \mathbf{resp}_{\text{OR}} = (\mathbf{ch}_0, \mathbf{resp}_0, \mathbf{resp}_1)$. Then, the verifier algorithm of ID_{OR}[ID] is also represented as in Eq.(2) by using f_{OR}^V . It means that $\mathbf{com}$ can be eliminated in the signature of $\text{SIG}[\text{ID}_{\text{OR}}[\text{ID}], \mathbf{H}]$.

Couveignes-Stolbunov identification scheme. To instantiate the Parallel-OR identification scheme from group action, we will use Couveignes-Stolbunov identification scheme ID_{CouSto} [Cou06, BKV19, Sto12], depicted in Figure 5. (The details of group action are provided in Appendix B.) Let GAGen be an efficient algorithm that generates a description of group action. The original protocol has 1 bit challenge. To extend it to t bits challenge, we simply repeat the protocol t times.

For completeness, we show that ID_{CouSto} has correctness, high commitment entropy, single secret keys, perfect HVZK, strong special soundness, the key recovery resistance in the multi-user setting, and random self-reducible under the group action discrete logarithm (GADL) assumption.

Theorem 11. *ID_{CouSto} has correctness, $t \log|\mathcal{G}|$ -bits of commitment min-entropy, and a single secret key.*

Proof. The correctness of ID_{CouSto} is clear. The commitment of ID_{CouSto} consists of t random elements over $\mathcal{G}$. Thus, ID_{CouSto} has $t \log|\mathcal{G}|$ -bits of commitment min-entropy. Also, since the group action is regular, for each $\mathbf{pk}$, there exists a unique secret key $\mathbf{sk}$. Thus, ID_{CouSto} has a single secret key. $\square$

ISetup_{CouSto}(1^λ)	P_{CouSto,1}(pk, sk)
1 : $(\mathcal{G}, \mathcal{E}, E_0, \star) \leftarrow \text{GAGen}(1^\lambda)$	1 : foreach $i \in [t]$ do
2 : $\text{ChSet} := \{0, 1\}^t$	2 : $b_i \leftarrow \mathcal{G}$
3 : return $\text{par} := (\mathcal{G}, \mathcal{E}, E_0, \star)$	3 : $\hat{E}_i := b_i \star E_0$
IGen_{CouSto}(par)	4 : com $:= (\hat{E}_1, \dots, \hat{E}_t)$
1 : $a_1 \leftarrow \mathcal{G}$	5 : st $:= (\text{sk}, (b_1, \dots, b_t))$
2 : $\text{sk} := a_1$	6 : return (com, st)
3 : $\text{pk} := E_1 = a_1 \star E_0$	
4 : return (pk, sk)	P_{CouSto,2}(ch, st)
V_{CouSto}(pk, com, ch, resp)	1 : $(a_1, (b_1, \dots, b_t)) := \text{st}$
1 : if com = $(\text{resp}_i \star E_{\text{ch}_i})_{i \in [t]}$ then	2 : $a_0 := 1_{\mathcal{G}}$
2 : return 1	3 : $(\text{ch}_1, \dots, \text{ch}_t) := \text{ch}$
3 : return 0	4 : foreach $i \in [t]$ do
	5 : $r_i := b_i \cdot a_{\text{ch}_i}^{-1}$
	6 : return $\text{resp} := (r_1, \dots, r_t)$

Fig. 5: The Couveignes-Stolbunov identification scheme $\text{ID}_{\text{CouSto}}$.

Theorem 12. $\text{ID}_{\text{CouSto}}$ is strong special sound. More precisely, $\text{Adv}_{\mathcal{A}, \text{ID}_{\text{CouSto}}[\text{ID}]}^{\text{SSS}}(\lambda) = 0$ for all (including computationally unbounded) adversaries $\mathcal{A}$.

Proof. We define the extractor Ext as follows. On input $(\text{pk}, \text{com}, \text{ch}, \text{resp}, \text{ch}', \text{resp}')$, if $\text{ch} = \text{ch}'$, Ext outputs $\perp$. If $\text{ch} \neq \text{ch}'$, there must exist an index I such that the I 'th bit of them are different, i.e., $\{\text{ch}_I, \text{ch}'_I\} = \{0, 1\}$. For such an I , Ext outputs $(\text{resp}'_I \cdot (\text{resp}_I)^{-1})^{\text{ch}_I - \text{ch}'_I}$.

It is sufficient to show that Ext can extract a_1 when $(\text{ch}, \text{resp}) \neq (\text{ch}', \text{resp}')$ and both $(\text{pk}, \text{com}, \text{ch}, \text{resp})$ and $(\text{pk}, \text{com}, \text{ch}', \text{resp}')$ are accepted by V_{CouSto} . Further, from the regularity of group action and the fact that com and ch uniquely determine resp , we can assume $\text{ch} \neq \text{ch}'$.

When $\text{ch} \neq \text{ch}'$, define I as above. Since $E_b = a_1^b \star E_0$ holds for both bit $b \in \{0, 1\}$, we have $\text{com}_I = \text{resp}_I \star E_{\text{ch}_I} = (\text{resp}_I \cdot a_1^{\text{ch}_I}) \star E_0$ and $\text{com}_I = (\text{resp}'_I \cdot a_1^{\text{ch}'_I}) \star E_0$. Thus, we have $(\text{resp}'_I \cdot (\text{resp}_I)^{-1})^{\text{ch}_I - \text{ch}'_I} = (a_1^{\text{ch}_I - \text{ch}'_I})^{\text{ch}_I - \text{ch}'_I} = a_1$. Therefore, Ext succeeds in extracting a_1 as desired. $\square$

Theorem 13. $\text{ID}_{\text{CouSto}}$ is perfect HVZK.

Proof. Consider the following simulator: On input $(\text{pk} = E_1, \text{ch})$, choose $\text{resp}_i \leftarrow \mathcal{G}$, compute $\text{com}_i := \text{resp}_i \star E_{\text{ch}_i}$ for all $i \in [t]$, and output $\text{com} = (\text{com}_1, \dots, \text{com}_t)$ and $\text{resp} = (\text{resp}_1, \dots, \text{resp}_t)$.

It is easy to confirm that the simulator's output has the same distribution as a real transcript between an honest prover and an honest verifier. $\square$

Theorem 14. Under the GADL assumption, $\text{ID}_{\text{CouSto}}$ satisfies the key recovery resistance. In particular, if there is an adversary $\mathcal{A}$ that breaks the key recovery resistance in time $T_{\mathcal{A}}$ with success probability $\text{Adv}_{\mathcal{A}, \text{ID}_{\text{CouSto}}}^{\text{KR}}(\lambda)$, then there is an algorithm $\mathcal{B}$ solving GADL problems in time $T_{\mathcal{B}} = T_{\mathcal{A}}$ with probability $\text{Adv}_{\mathcal{B}, \text{GADL}}^{\text{GADL}}(\lambda) = \text{Adv}_{\mathcal{A}, \text{ID}_{\text{CouSto}}}^{\text{KR}}(\lambda)$.

Proof. Let $\mathcal{A}$ be an adversary that breaks the key recovery resistance. Consider the following GADL solver $\mathcal{B}$ that uses $\mathcal{A}$ as a subroutine: Upon receiving a GADL instance $(\mathcal{G}, \mathcal{E}, E_0, \star, E_1)$, $\mathcal{B}$ sets $\text{par} := (\mathcal{G}, \mathcal{E}, E_0, \star)$ and $\text{pk} := E_1$. $\mathcal{B}$ executes $\mathcal{A}$ on input (par, pk) and receives $\text{sk} \in \mathcal{G}$ from $\mathcal{A}$. $\mathcal{B}$ outputs sk as the solution of the GADL instance.

We can verify that if $\mathcal{A}$ breaks the key recovery resistance, $\mathcal{B}$ solves the given GADL instance since $\mathcal{A}$'s output sk satisfies $\text{pk} = \text{sk} \star E_0$, meaning that sk is the GADL of $\text{pk} = E_1$ w.r.t. E_0 . Therefore, we have $\text{Adv}_{\mathcal{B}, \text{GAGen}}^{\text{GADL}}(\lambda) = \text{Adv}_{\mathcal{A}, \text{ID}_{\text{CouSto}}}^{\text{KR}}(\lambda)$. Note that the running time of $\mathcal{B}$ is $T_{\mathcal{B}} = T_{\mathcal{A}}$ since $\mathcal{B}$ executes $\mathcal{A}$ once. $\square$

Theorem 15. $\text{ID}_{\text{CouSto}}$ is random self-reducible.

Proof. ReRand and DeRand are defined as follows:

ReRand(par, pk): Let $E_1 := \text{pk}$. Choose $c_1 \leftarrow \mathcal{G}$ and output $\text{pk}' := c_1 \star E_1$ and $\text{td}' := c_1$.
DeRand($\text{pk}, \text{pk}', \text{sk}', \text{td}'$): Let $a'_1 := \text{sk}'$ and $c_1 := \text{td}'$. Output $\text{sk}^* := a'_1 \cdot (c_1)^{-1}$.

We have that, for all $(\text{pk}, \text{sk}) \in \text{IGen}(\text{par})$, pk' output from ReRand(par, pk) is uniformly distributed and has the same distribution as a freshly-generated key pair. Also, for all $(\text{pk}', \text{td}') \leftarrow \text{ReRand}(\text{par}, \text{pk})$ and $\text{sk}^* \leftarrow \text{DeRand}(\text{pk}, \text{pk}', \text{sk}', \text{td}')$, if $\text{pk}' = a'_1$ is a valid secret key of $\text{pk}' = c_1 \star E_1$, we have $\text{sk}^* \star E_0 = (a'_1 \cdot (c_1)^{-1}) \star E_0 = (c_1)^{-1} \star (a'_1 \star E_0) = (c_1)^{-1} \star (c_1 \star E_1) = E_1$. Therefore, sk^* is a valid secret key of $\text{pk} = E_1$. $\square$

By instantiating $\text{SIG}[\text{ID}, \text{H}]$ with $\text{ID} = \text{ID}_{\text{OR}}[\text{ID}_{\text{CouSto}}]$, we obtain a signature scheme, $\text{SIG}[\text{ID}_{\text{OR}}[\text{ID}_{\text{CouSto}}], \text{H}]$, whose MU-SUF-CMA-C security is tightly implied from the GADL assumption.

Corollary 4. Under the GADL assumption, $\text{SIG}[\text{ID}_{\text{OR}}[\text{ID}_{\text{CouSto}}], \text{H}]$ is MU-SUF-CMA-C secure in the non-programmable random oracle model. In particular, for any adversary $\mathcal{A}$ that breaks the MU-SUF-CMA-C security of $\text{SIG}[\text{ID}_{\text{OR}}[\text{ID}_{\text{CouSto}}], \text{H}]$ in time $T_{\mathcal{A}}$ with probability $\text{Adv}_{\mathcal{A}, \text{SIG}[\text{ID}_{\text{OR}}[\text{ID}_{\text{CouSto}}], \text{H}]}^{\text{MU-SUF-CMA-C}}(\lambda)$, there is an algorithm $\mathcal{B}$ solving a GADL problem in time $T_{\mathcal{B}} = \mathcal{O}(T_{\mathcal{A}})$ with probability $\text{Adv}_{\mathcal{B}, \text{GAGen}}^{\text{GADL}}(\lambda)$ such that

$$\begin{aligned} & \text{Adv}_{\mathcal{A}, \text{SIG}[\text{ID}_{\text{OR}}[\text{ID}_{\text{CouSto}}], \text{H}]}^{\text{MU-SUF-CMA-C}}(\lambda) \\ & \leq 2\text{Adv}_{\mathcal{B}, \text{GAGen}}^{\text{GADL}}(\lambda) + \frac{Q_{\text{RO}} + 1}{2^{\rho t \log |\mathcal{G}|}} + \frac{T \cdot Q_{\text{sig}}(Q_{\text{RO}} + T \cdot Q_{\text{sig}})}{2^{\rho \gamma}}. \end{aligned}$$

5 Multi-user Security of Hash-Based Signature

As a complementary study, this section analyzes the MU-SUF-CMA-C security of hash-based signature schemes.

5.1 Our Target: Merkle Signature with Hyper-Tree Extension

We will analyze the Merkle signature scheme (MSS) [Mer79, Mer90] with the hyper-tree extension [BDK⁺07], which is the core framework of standardized hash-based signature schemes such as SPHINCS+ [HBD⁺22]. Let $\text{OTS} = (\text{OTS.Setup}, \text{OTS.KGen}, \text{OTS.Sign}, \text{OTS.Vf})$ be a one-time signature, such as the Lamport signature. We assume that the random coins rand used by OTS.KGen can be easily determined by the generated secret key osk .⁷ Let $\text{H}_{\text{cr}} : \{0, 1\}^* \rightarrow \{0, 1\}^{2\lambda}$ be a collision resistant hash function and let $\text{PRF} : \mathcal{S} \times \{0, 1\}^* \rightarrow \mathcal{R}$ be a pseudo-random function, where $\text{rand} \in \mathcal{R}$ is a random coin of OTS.KGen . Let us define three algorithms related to the Merkle tree that use H_{cr} as a hash function:

- $\text{MT.Gen}((\text{node}_i)_{i \in \{0, 1\}^L})$ constructs a Merkle tree from 2^L leaf nodes and outputs the root hash root .
- $\text{MT.Auth}((\text{node}_i)_{i \in \{0, 1\}^L}, r)$ constructs a Merkle tree and outputs root as $\text{MT.Gen}((\text{node}_i)_{i \in \{0, 1\}^L})$ does, but also outputs the authentication path auth related to a specified node r .
- $\text{MT.Recon}(r, \text{node}_r, \text{auth})$ reconstructs the Merkle tree from a node node_r and the corresponding authentication path auth and outputs the root hash root .

The hash-based signature scheme $\text{HSIG}[\text{OTS}, \text{H}_{\text{cr}}, \text{PRF}]$ is depicted in Figure 6. L denotes the height of an inner tree (i.e., the height of each Merkle tree) and D denotes the height of the hyper-tree (i.e., the height of the tree of the Merkle trees). They should satisfy $L \cdot D = \mathcal{O}(\lambda)$ to allow for stateless signing.

⁷ In most cases, $\text{osk} = \text{rand}$ holds.

Setup(1^λ)	Sign(ssk, m)
1 : $\text{par}' \leftarrow \text{OTS.Setup}(1^\lambda)$	1 : $r_1, \dots, r_D \leftarrow \{0, 1\}^L$
2 : return $\text{par} := \text{par}'$	2 : for $d = D, \dots, 1$ do
KGen (par)	3 : foreach $i \in \{0, 1\}^L$ do
1 : $\text{seed} \leftarrow \mathcal{S}$	4 : $\text{str} := r_1 \parallel \dots \parallel r_{d-1} \parallel i$
2 : foreach $i \in \{0, 1\}^L$	5 : $\text{rand}_{d,i} \leftarrow \text{PRF}(\text{seed}, \text{str})$
3 : $\text{rand}_{1,i} \leftarrow \text{PRF}(\text{seed}, i)$	6 : $(\text{osk}_{d,i}, \text{ovk}_{d,i})$
4 : $(\text{ovk}_{1,i}, \text{osk}_{1,i})$	$\leftarrow \text{OTS.KGen}(\text{par}; \text{rand}_{d,i})$
$\leftarrow \text{OTS.KGen}(\text{par}; \text{rand}_{1,i})$	7 : endfor
5 : endfor	8 : $(\text{root}_d, \text{auth}_d)$
6 : $\text{root}_1 \leftarrow \text{MT.Gen}((\text{ovk}_{1,i})_{i \in \{0,1\}^L})$	$\leftarrow \text{MT.Auth}((\text{ovk}_{d,i})_{i \in \{0,1\}^L}, r_d)$
7 : return $(\text{svk}, \text{ssk}) := (\text{root}_1, \text{seed})$	9 : if $d = D$ then $m_d := m$
Vf (svk, m , σ)	10 : else $m_d := \text{root}_{d+1}$
1 : parse $\sigma = (r_d, \sigma_d, \text{ovk}_d, \text{auth}_d)_{d \in [D]}$	11 : $\sigma_d \leftarrow \text{OTS.Sign}(\text{osk}_{d,r_d}, m_d)$
2 : for $d = D, \dots, 1$ do	12 : endfor
3 : if $d = D$ then $m_d := m$	13 : $\sigma := (r_d, \sigma_d, \text{ovk}_{d,r_d}, \text{auth}_d)_{d \in [D]}$
4 : else $m_d := \text{root}_{d+1}$	14 : return σ
5 : if $\text{OTS.Vf}(\text{ovk}_d, m_d, \sigma_d) = 0$ then	
6 : return 0	
7 : $\text{root}_d \leftarrow \text{MT.Recon}(r_d, \text{ovk}_d, \text{auth}_d)$	
8 : endfor	
9 : if $\text{root}_1 \neq \text{svk}$ then return 0	
10 : return 1	

Fig. 6: Hash-based signature scheme $\text{HSIG}[\text{OTS}, \text{H}_{\text{cr}}, \text{PRF}]$.

5.2 Security Analysis

To analyze the multi-user security of $\text{HSIG}[\text{OTS}, \text{H}_{\text{cr}}, \text{PRF}]$, we first consider its modified version $\text{HSIG}^-[\text{OTS}, \text{H}_{\text{cr}}]$ in which a PRF is never used and all random coins rand are randomly chosen in the key generation process and stored in ssk . Note that ssk consists of $\sum_{d=1}^D (2^L)^d < 2 \cdot 2^L D$ of rand , i.e., $\text{ssk} = \{\text{rand}_{\text{str}} | \text{str} \in \{0, 1\}^L \cup (\{0, 1\}^L)^2 \cup \dots \cup (\{0, 1\}^L)^D\}$. Thus, we assume $L \cdot D$ is not $O(\lambda)$ but D and L are constants, otherwise the key generation algorithm becomes super-polynomial.

As the following theorem states, the multi-user security of $\text{HSIG}^-[\text{OTS}, \text{H}_{\text{cr}}]$ can be tightly reduced to the collision resistance of H_{cr} in the standard model.

Lemma 1. *If OTS is one-time MU-SUF-CMA-C (ot-MU-SUF-CMA-C) secure and H_{cr} is a collision resistant hash function, then $\text{HSIG}^-[\text{OTS}, \text{H}_{\text{cr}}]$ is MU-SUF-CMA-C secure in the standard model.*

In particular, if there is an adversary $\mathcal{A}$ that breaks the MU-SUF-CMA-C security of $\text{HSIG}^-[\text{OTS}, \text{H}_{\text{cr}}]$ with success probability $\text{Adv}_{\mathcal{A}, \text{HSIG}^-[\text{OTS}, \text{H}_{\text{cr}}]}^{\text{MU-SUF-CMA-C}}(\lambda)$, then there is an algorithm $\mathcal{B}_1$ breaking the one-time MU-SUF-CMA-C of OTS with probability $\text{Adv}_{\mathcal{B}_1, \text{OTS}}^{\text{ot-MU-SUF-CMA-C}}(\lambda)$ and an algorithm $\mathcal{B}_2$ breaking the collision resistance of H_{cr} with probability $\text{Adv}_{\mathcal{B}_2, \text{H}_{\text{cr}}}^{\text{CR}}(\lambda)$ such that

$$\text{Adv}_{\mathcal{A}, \text{HSIG}[\text{OTS}, \text{H}_{\text{cr}}, \text{PRF}]}^{\text{MU-SUF-CMA-C}}(\lambda) \leq \text{Adv}_{\mathcal{B}_1, \text{OTS}}^{\text{ot-MU-SUF-CMA-C}}(\lambda) + \text{Adv}_{\mathcal{B}_2, \text{H}_{\text{cr}}}^{\text{CR}}(\lambda). \quad (3)$$

Proof (sketch). Let $\mathcal{A}$ be an adversary against the MU-SUF-CMA-C security of $\text{HSIG} = \text{HSIG}^-[\text{OTS}, \text{H}_{\text{cr}}]$, u^* be the (uncorrupted) target user, and (m^*, σ^*) be the forged message-signature pair. For a forged signature $\sigma^* = (r_d^*, \sigma_d^*, \text{ovk}_d^*, \text{auth}_d^*)_{d \in [D]}$, w.l.o.g, we assume that the signing oracle has returned an honestly-generated signature $\sigma' = (r'_d, \sigma'_d, \text{ovk}'_d, \text{auth}'_d)_{d \in [D]} \leftarrow \text{HSIG}.\text{Sign}(\text{ssk}^{u^*}, m')$ to $\mathcal{A}$ such that $(r'_1, \dots, r'_D) = (r_1^*, \dots, r_D^*)$ and $(m', \sigma') \neq (m^*, \sigma^*)$. (If not, the game can generate it after receiving the forged signature.) Based on σ^* and σ' , we define $\text{root}_d^* := \text{MT}.\text{Recon}(r_d^*, \text{ovk}_d^*, \text{auth}_d^*)$, $\text{root}'_d := \text{MT}.\text{Recon}(r'_d, \text{ovk}'_d, \text{auth}'_d)$ for $d \in [D]$ and following events.

- (A-1) $\text{root}_1^* = \text{root}'_1 \wedge (\text{ovk}_1^*, \text{auth}_1^*) \neq (\text{ovk}'_1, \text{auth}'_1)$
- (A-2) $1 < \exists d \leq D : (\text{root}_d^*, \sigma_{d-1}^*) = (\text{root}'_d, \sigma'_{d-1}) \wedge (\text{ovk}_d^*, \text{auth}_d^*) \neq (\text{ovk}'_d, \text{auth}'_d)$
- (B-1) $1 \leq \exists d < D : (\text{root}_{d+1}^*, \sigma_d^*) \neq (\text{root}'_{d+1}, \sigma'_d) \wedge (\text{ovk}_d^*, \text{auth}_d^*) = (\text{ovk}'_d, \text{auth}'_d)$
- (B-2) $(m^*, \sigma_D^*) \neq (m', \sigma'_D) \wedge \text{ovk}_D^* = \text{ovk}'_D$
- (A) = (A-1) $\vee$ (A-2), (B) = (B-1) $\vee$ (B-2)

It is easy to see that, if neither of (A) nor (B) occurs, $\mathcal{A}$ fails to forge a signature. Further, (A) implies breaking the collision resistance of H_{cr} . Thus we have

$$\begin{aligned} & \text{Adv}_{\mathcal{A}, \text{HSIG}[\text{OTS}, \text{H}_{\text{cr}}, \text{PRF}]}^{\text{MU-SUF-CMA-C}}(\lambda) \\ & \leq \Pr[(\text{A}) \text{ occurs}] + \Pr[N\text{-ot-MU-SUF-CMA-C} \Rightarrow 1 \wedge (\text{B}) \text{ occurs}] \\ & \leq \text{Adv}_{\mathcal{B}_2, \text{H}_{\text{cr}}}^{\text{CR}}(\lambda) + \Pr[N\text{-ot-MU-SUF-CMA-C} \Rightarrow 1 \wedge (\text{B}) \text{ occurs}] \end{aligned}$$

for some $\mathcal{B}_2$.

Next, we will evaluate the second term and show the next equation.

$$\begin{aligned} & \Pr[N\text{-ot-MU-SUF-CMA-C} \Rightarrow 1 \wedge (\text{B}) \text{ occurs}] \\ & \leq \text{Adv}_{\mathcal{B}_1, \text{OTS}}^{\text{ot-MU-SUF-CMA-C}}(\lambda) \end{aligned} \quad (4)$$

To this end, we construct $\mathcal{B}_1$ as follows. $\mathcal{B}_1$ receives N' verification keys $\{\text{ovk}_{u'}^{**}\}_{u' \in [N']}$ from its challenger, and simulates the attack game as follows.

- $\mathcal{B}_1$ does not choose $\text{ssk}^u = \{\text{rand}_{\text{str}}^u\}$ for all users $u \in [N]$ at the beginning of the game.
- $\mathcal{B}_1$ does not compute $\text{OTS}.\text{KGen}$ throughout the game. Instead, $\mathcal{B}_1$ assigns one $\text{ovk}_{u'}^{**}$ to each $\text{ovk}_{\text{str}}^u$.
- When user u is corrupted, $\mathcal{B}_1$ issues corruption queries for all users u' such that $\text{ovk}_{u'}^{**}$ was assigned to $\text{ovk}_{\text{str}}^u$ for some str . Upon obtaining $\text{osk}_{u'}^{**}$, $\mathcal{B}_1$ finds $\text{rand}_{u'}^{**}$ used to generate $\text{osk}_{u'}^{**}$, and returns ssk^u that consists of appropriate $\text{rand}_{u'}^{**}$.

At the end of the game, if $\mathcal{A}$ succeeds in forging and (B-2) defined above occurs, $\text{ovk}_D^* = \text{ovk}'_D = \text{ovk}_{u'}^{**}$ holds for some u' . Thus $\mathcal{B}_1$ succeeds in forging by outputting (u', m^*, σ_D^*) . If $\mathcal{A}$ succeeds in forging and (B-1) defined above occurs, $\text{ovk}_d^* = \text{ovk}'_d = \text{ovk}_{u'}^{**}$ holds for some u' and σ_{d+1}^* is a valid signature of message $m_{d+1}^* := \text{root}_d^*$ under the verification key $\text{ovk}_{u'}^{**}$. Thus $\mathcal{B}_1$ succeeds in forging by outputting $(u', m_{d+1}^*, \sigma_{d+1}^*)$. Consequently, $\mathcal{B}_1$ can succeed in forging whenever $\mathcal{A}$ succeeds and (B)=(B-1) $\wedge$ (B-2) occurs. Thus, we have Equation (4). $\square$

By slightly modifying the above proof, the multi-user security of $\text{HSIG}[\text{OTS}, \text{H}_{\text{cr}}, \text{PRF}]$ can be shown as follows.

Theorem 16. *If OTS is one-time MU-SUF-CMA-C (ot-MU-SUF-CMA-C) secure and H_{cr} is a collision resistant hash function, then $\text{HSIG}[\text{OTS}, \text{H}_{\text{cr}}, \text{PRF}]$ is MU-SUF-CMA-C secure in the programmable random oracle model.*

In particular, if there is an adversary $\mathcal{A}$ that breaks the MU-SUF-CMA-C security of $\text{HSIG}[\text{OTS}, \text{H}_{\text{cr}}, \text{PRF}]$ in time $T_{\mathcal{A}}$ with success probability $\text{Adv}_{\mathcal{A}, \text{HSIG}[\text{OTS}, \text{H}_{\text{cr}}, \text{PRF}]}^{\text{MU-SUF-CMA-C}}(\lambda)$, then there is an algorithm $\mathcal{B}'_1$ breaking the one-time MU-SUF-CMA-C of OTS in time $T_{\mathcal{B}'_1} = \mathcal{O}(T_{\mathcal{A}})$ with probability $\text{Adv}_{\mathcal{B}'_1, \text{OTS}}^{\text{ot-MU-SUF-CMA-C}}(\lambda)$ and an algorithm $\mathcal{B}_2$ breaking the collision resistance of H_{cr} in time $T_{\mathcal{B}_2} = \mathcal{O}(T_{\mathcal{A}})$ with probability $\text{Adv}_{\mathcal{B}_2, \text{H}_{\text{cr}}}^{\text{CR}}(\lambda)$ such that

$$\text{Adv}_{\mathcal{A}, \text{HSIG}[\text{OTS}, \text{H}_{\text{cr}}, \text{PRF}]}^{\text{MU-SUF-CMA-C}}(\lambda) \leq \text{Adv}_{\mathcal{B}'_1, \text{OTS}}^{\text{ot-MU-SUF-CMA-C}}(\lambda) + \text{Adv}_{\mathcal{B}_2, \text{H}_{\text{cr}}}^{\text{CR}}(\lambda) + \frac{Q_{\text{RO}} \cdot Q_{\text{corr}}}{2^{|S|}}. \quad (5)$$

Here, Q_{RO} and Q_{corr} are the maximum number of RO and corruption queries issued by $\mathcal{A}$, respectively.

Proof (sketch). The proof is very similar to that of Lemma 1 except Equation (4) is replaced with

$$\begin{aligned} & \Pr[N\text{-ot-MU-SUF-CMA-C} \Rightarrow 1 \wedge (B) \text{ occurs}] \\ & \leq \text{Adv}_{\mathcal{B}'_1, \text{OTS}}^{\text{ot-MU-SUF-CMA-C}}(\lambda) + \frac{Q_{\text{RO}} \cdot Q_{\text{corr}}}{2^{|S|}}. \end{aligned} \quad (6)$$

The construction of $\mathcal{B}'_1$ is similar to $\mathcal{B}_1$, but in this case, each rand_{str} must equal to $\text{PRF}(\text{seed}, str)$. More precisely,

- $\mathcal{B}'_1$ does not compute $\text{PRF}(\text{seed}^u, str)$ nor OTS.KGen throughout the game, and assigns one $\text{ovk}_{u'}^{**}$ to each ovk_{str}^u instead. Note that the number of one-time keys ovk_{str}^u used to simulate OTS.KGen and OTS.Sig is at most $N' = 2^L(N + (D - 1)Q_{\text{Sign}})$; $2^L N$ keys to generate verification keys for N users, at most $2^L(D - 1)$ additional keys to answer each signing query.
- PRF is modeled as a programmable random oracle. $\mathcal{B}'_1$ answers a random value rand for each random oracle query.
- When user u is corrupted by $\mathcal{A}$, $\mathcal{B}'_1$ first chooses a random $\text{seed}^u \leftarrow \mathcal{S}$. If $\mathcal{A}$ has already queried (seed^u, str) to the random oracle for some str , $\mathcal{B}'_1$ aborts the game. Otherwise, $\mathcal{B}'_1$ issues corruption queries to obtain $\text{osk}_{u'}^{**}$ and sends the appropriate ssk^u to $\mathcal{A}$ as $\mathcal{B}_1$ does. In addition, $\mathcal{B}'_1$ programs the random oracle as $\text{PRF}(\text{seed}^u, str) := \text{rand}_{u'}^{**}$ for all str s.t. $\text{ovk}_{u'}^{**} = \text{ovk}_{str}^u$.

$\mathcal{B}'_1$ can succeed in forging whenever it does not abort the game, $\mathcal{A}$ succeeds, and (B) occurs. Thus we have Equation (6) from

$$\Pr[\mathcal{B}_1 \text{ aborts the game}] \leq \frac{Q_{\text{RO}} \cdot Q_{\text{corr}}}{2^{|S|}}.$$

□

Remark 5. In the construction of $\mathcal{B}'_1$ in the above proof, it is essential to program a random oracle to respond to corruption queries, and it seems impossible to prove the security in the non-programmable random oracle model (or the standard model) as long as we use PRFs. The only solution to avoid the programmability (or the random oracle model itself) that we are aware of is to use $\text{HSIG}^-(\text{OTS}, \text{H}_{\text{cr}})$ as a stateless scheme. Namely, a signer maintains a counter $r = r_1 \| r_2 \| \dots \| r_D$ and records all key pairs of OTS that are necessary for future signing. Since r is used in order unlike in stateless schemes, only a polynomial number of key pairs of OTS need to be maintained at any moment even if $LD = O(\lambda)$.

Remark 6. If we model not only the PRF but also the hash function as a random oracle, the collision resistance of hash functions is not needed to be explicitly assumed, since it is implied from properties of the random oracle. In contrast, the above theorem minimizes the dependence on the (unrealistic) power of the random oracle by assuming the collision resistance.

5.3 Instantiation of OTS: Variant of Lamport Signature

As a one-time MU-SUF-CMA-C signature scheme, we consider a variant of Lamport signature, denoted by OTS^{Lam} , defined as follows. (Concrete algorithms are depicted in Figure 7.)

Let $\text{H}_{\text{cr}} : \{0, 1\}^* \rightarrow \{0, 1\}^{2\lambda}$ be a hash function. As in the original Lamport scheme, the signing key osk of OTS^{Lam} consists of 2λ pairs of random strings $(x_{0,j}, x_{1,j})$, the corresponding verification key ovk consists of 2λ pairs of $(\text{H}_{\text{cr}}(x_{0,j}), \text{H}_{\text{cr}}(x_{1,j}))$, but each $x_{b,j}$ is 4λ bits, while each hash value $\text{H}_{\text{cr}}(x_{b,j})$ is 2λ bits. To support arbitrary-length messages, a message m to be signed is first hashed, and the signature $\sigma = (x_{m,j})_{j \in [2\lambda]}$ is generated based on the message digest $(m_1 \| \dots \| m_{2\lambda}) = \text{H}_{\text{cr}}(m)$. We note that there are many osk 's corresponding to the same ovk . One valid signature reveals the values of 2λ of the $x_{b,j}$'s, but even an unbounded adversary cannot determine the correct values of the remaining $x_{b,j}$.

It is easy to show that the multi-user security with adaptive corruption of OTS^{Lam} is tightly reduced to the collision resistance of H_{cr} . The reduction $\mathcal{B}$ generates N key pairs by itself and runs the adversary $\mathcal{A}$.

Setup(1^λ)	Sign(osk, m)
1 : return par := 1^λ	1 : $m_1 \parallel \dots \parallel m_{2\lambda} \leftarrow H_{cr}(m)$
KGen(par)	2 : $\sigma := (x_{m_j, j})_{j \in [2\lambda]}$
1 : foreach $j \in [2\lambda]$ do	3 : return σ
2 : $x_{0,j}, x_{1,j} \leftarrow \{0, 1\}^{4\lambda}$	Vf(ovk, m, σ)
3 : $y_{0,j} \leftarrow H_{cr}(x_{0,j})$	1 : $m_1 \parallel \dots \parallel m_{2\lambda} \leftarrow H_{cr}(m)$
4 : $y_{1,j} \leftarrow H_{cr}(x_{1,j})$	2 : $(x_j)_{j \in [2\lambda]} := \sigma$
5 : endfor	3 : foreach $j \in [2\lambda]$ do
6 : osk := $(x_{0,j}, x_{1,j})_{j \in [2\lambda]}$	4 : if $H_{cr}(x_j) \neq y_{m_j, j}$ then
7 : ovk := $(y_{0,j}, y_{1,j})_{j \in [2\lambda]}$	5 : return 0
8 : return (ovk, osk)	6 : endfor
	7 : return 1

Fig. 7: A variant of Lamport signature scheme OTS^{Lam} .

Since $\mathcal{B}$ knows all secret keys, it can answer any corruption queries and signature queries. Assume that $\mathcal{A}$ succeeds in (strongly) forging, i.e., finding an acceptable message-signature pair $(m^*, \sigma^* = (x_j^*)_{j \in [2\lambda]})$ for the user i^* upon knowing one valid message-signature pair $(m', \sigma' = (x'_j)_{j \in [2\lambda]})$, where $(m^*, \sigma^*) \neq (m', \sigma')$. Let $(m_1^* \parallel \dots \parallel m_{2\lambda}^*) := H_{cr}(m^*)$ and $(m'_1 \parallel \dots \parallel m'_{2\lambda}) := H_{cr}(m')$.

- If $m^* \neq m'$ and $(m_1^* \parallel \dots \parallel m_{2\lambda}^*) = (m'_1 \parallel \dots \parallel m'_{2\lambda})$, then (m^*, m') is an immediate collision pair.
- If $m^* \neq m'$ and $(m_1^* \parallel \dots \parallel m_{2\lambda}^*) \neq (m'_1 \parallel \dots \parallel m'_{2\lambda})$, then there exists j such that $m_j^* \neq m'_j$. For such j , x_j^* in the forged signature satisfies $H_{cr}(x_j^*) = H_{cr}(x_{m_j^*, j})$. Since the uncorrupted and unused secret key $x_{m_j^*, j}$ is perfectly hidden from $\mathcal{A}$, $x_j^* \neq x_{m_j^*, j}$ holds with overwhelming probability. So, $(x_j^*, x_{m_j^*, j})$ is a collision pair.
- If $m^* = m'$, there exists j such that $x_j^* \neq x'_j$. Since both σ^* and σ' are accepted, $H_{cr}(x_j^*) = H_{cr}(x_{m_j^*, j}) = H_{cr}(x_{m'_j, j}) = H_{cr}(x'_j)$ holds. Therefore, (x_j^*, x'_j) is a collision pair.

Therefore, if $\mathcal{A}$ creates a valid forgery without knowing the target user's secret key osk_{i^*} , $\mathcal{B}$ can break the collision resistance of H_{cr} ,

Consequently, we obtain the following corollary.

Corollary 5. *The MU-SUF-CMA-C security of $HSIG[OTS^{Lam}, H_{cr}, PRF]$ can be tightly reduced to the collision resistance of H_{cr} in the programmable random oracle model.*

Remark 7. Winternitz signature can be used to shorten the signature length. However, its iterative hashing prevents the application of the above proof technique. We note that Katz [Kat16] analyzed the multi-user security (without corruption) of Leighton and Micali one-time signature scheme [MCF19] (a variant of Winternitz signature) by modeling the hash function as a random oracle.⁸ We observe that the proof in [Kat16] also works when adaptive corruptions are considered. Therefore, from his result and our result (Theorem 16), tight multi-user security (with adaptive corruption) of the Hierarchical Signature System (HSS) [MCF19] can be shown in the random oracle model.

⁸ Eaton also provided similar analysis in the QROM [Eat17], but it seems difficult to extend his analysis in the multi-user setting with corruption.

Table 2: Efficiency of $\text{SIG}[\text{ID}_{\text{Oka}}, \text{H}]$ in 128-bit security. We set $T = 3$.

ρ	γ	t	β	#hash	svk	\sigma
32	4	9	2^{-123}	512	32 B	2.1 KB
22	6	11	2^{-123}	1241	32 B	1.4 KB
16	8	13	2^{-126}	4096	32 B	1.1 KB

Table 3: Efficiency of $\text{SIG}[\text{ID}_{\text{OR}}[\text{ID}_{\text{CouSto}}], \text{H}]$ with CSIDH-512 parameter. We set $T = 3$.

ρ	γ	t	β	#hash	svk	\sigma
32	4	9	2^{-123}	512	128 B	18.5 KB
22	6	11	2^{-123}	1241	128 B	15.5 KB
16	8	13	2^{-126}	4096	128 B	13.4 KB

6 Efficiency Evaluation

In this section, we evaluate the efficiency of the concrete instantiations of the proposed signature scheme in Section 4. Also, for comparison, we provide the efficiency of the tightly-secure hash-based scheme discussed in Section 5.

6.1 Efficiency of Proposed Signature Scheme

We evaluate the efficiency of the classical group-based scheme $\text{SIG}[\text{ID}_{\text{Oka}}, \text{H}]$ and that of the isogeny-based instantiation derived from $\text{SIG}[\text{ID}_{\text{OR}}[\text{ID}_{\text{CouSto}}], \text{H}]$. Here, we consider 128-bit classical security (i.e., NIST Level-I).

Parameters for Fischlin transformation. For the choice of parameters used in randomized Fischlin transformation, we recall the recent result by Chen and Lindell [CL24] for the original version since their result is also valid for the randomized one. To achieve λ -bit security, the parameters must satisfy the following conditions: $\rho \cdot \gamma \geq \lambda$ and $2^{t-\gamma} - \log \rho \geq \lambda/T$. Chen and Lindell suggest that for any value of γ , for $\rho \leq 64$ set $t = \gamma + 5$ and for $\rho > 64$ set $t = \gamma + 6$. They also proved the probability of correctness errors, and the expected number of hash computations, summarized below:

- Probability of correctness error: $\beta = 2^{(-2^{t-\gamma} \log e + \log \rho)T}$.
- Expected number of hash computations: $\text{\#hash} = \rho \cdot 2^\gamma / (1 - \beta)$.

Parameters for classical groups and efficiency of $\text{SIG}[\text{ID}_{\text{Oka}}, \text{H}]$. For the choice of a multiplicative group, we can use the NIST P-256 curve with $\log|\mathbb{G}| = 256$, which achieves 128-bit security since $\text{SIG}[\text{ID}_{\text{Oka}}, \text{H}]$ is tightly secure. Thus, $|\text{resp}| = 512$ bits and the total signature size is $|\sigma| = \rho(t + |\text{resp}|) = \rho(t + 512)$ bits for 128-bit security. Note that we can remove **com** from the signature because ID_{Oka} 's verifier algorithm can be represented as follows. (See Section 3.2.)

$$V(\text{pk}, \text{com}, \text{ch}, (y_1, y_2)) = 1 \iff \text{com} = f_{\text{Oka}}^V(\text{pk}, \text{ch}, (y_1, y_2)) = g^{y_1} g_1^{y_2} / \text{pk}^{\text{ch}}.$$

We give in Table 2 the efficiency estimations for $\text{SIG}[\text{ID}_{\text{Oka}}, \text{H}]$ in 128-bit security in some parameter sets. As observed, there is a trade-off between the signature sizes and the signing times. Spending more computational cost on signature generation can shorten the signature size. The signature size of $\text{SIG}[\text{ID}_{\text{Oka}}, \text{H}]$ is about 1.5 KB, but the DDH-based tight MU-UF-CMA-C signature scheme [DGJL21, Sec. 5.1] has only 96 B signature. Thus, while $\text{SIG}[\text{ID}_{\text{Oka}}, \text{H}]$ relies on weaker DL assumption, the signature is about 15 times larger than the DDH-based scheme. This can be viewed as a trade-off between the strength of the assumptions on which security is based and the efficiency, but improving the efficiency of DL-based schemes is a future challenge.

Parameters for isogeny and efficiency of $\text{SIG}[\text{ID}_{\text{OR}}[\text{ID}_{\text{CouSto}}], \text{H}]$. We use the CSIDH-512 prime p and define the group action $g \star \mathcal{E}$ exactly as in CSI-FiSh [BKV19]. According to [BKV19, Sec. 2], the group

Table 4: Efficiency of $\text{HSIG}[\text{OTS}, \text{H}_{\text{cr}}, \text{PRF}]$ in 128-bit security. We set $L = D = 16$. w is Winternitz parameter.

OTS	#hash	svk	σ
OTS^{Lam}	1.0×10^{10}	32 B	533.0 KB
$\text{OTS}^{\text{LM}} (w = 1)$	2.7×10^8	32 B	140.3 KB
$\text{OTS}^{\text{LM}} (w = 2)$	4.1×10^8	32 B	74.8 KB
$\text{OTS}^{\text{LM}} (w = 4)$	1.0×10^9	32 B	42.5 KB
$\text{OTS}^{\text{LM}} (w = 8)$	1.0×10^{10}	32 B	29.2 KB

elements require roughly 256 bits, and elements in $\mathcal{E}$ require about 512 bits. Thus, the public key size is $2|\mathcal{E}| = 1024$ bits and the signature size is $|\sigma| = 2\rho t(|\mathcal{G}| + 1) = 514\rho t$ bits in the CSIDH-512 parameter set. Note that, similar to $\text{SIG}[\text{ID}_{\text{Oka}}, \text{H}]$, we can remove com's from the signature because $\text{SIG}[\text{ID}_{\text{OR}}[\text{ID}_{\text{CouSto}}], \text{H}]$'s verifier algorithm can be represented as follows. (See Section 3.2.)

$$\text{V}(\text{pk}, \text{com}, \text{ch}, \text{resp}) = 1 \iff \text{com} = f_{\text{CouSto}}^{\text{V}}(\text{pk}, \text{ch}, \text{resp}) = (\text{resp}_i \star E_{\text{ch}_i})_{i \in [t]}.$$

We give in Table 3 the efficiency estimations for $\text{SIG}[\text{ID}_{\text{OR}}[\text{ID}_{\text{CouSto}}], \text{H}]$ in some parameter sets. The concrete signature size of $\text{SIG}[\text{ID}_{\text{OR}}[\text{ID}_{\text{CouSto}}], \text{H}]$ instantiated with CSIDH-512 parameter is about 15 KB. On the other hand, the signature size of Pan and Wagner's group action-based scheme [PW22] with CSIDH-512 parameter is $2\lambda(|\mathcal{G}| + 1) = 8.2$ KB.⁹ While our signature is about two times larger than Pan and Wagner's signature, our scheme achieves tight MU-SUF-CMA-C security based on a weaker GADL assumption.

6.2 Efficiency of Hash-Based Signature

For completeness, we will evaluate the efficiency of $\text{HSIG}[\text{OTS}, \text{H}_{\text{cr}}, \text{PRF}]$ where OTS is instantiated by OTS^{Lam} (Section 5.3) and the LM-OTS signature (OTS^{LM}) [Kat16]. The verification key length is 2λ bits, and the signature size is $D(L(2\lambda + 1) + |\sigma_{\text{OTS}}| + |\text{ovk}|)$ bits because the size of *auth* is $L \cdot 2\lambda$ bits. As a metric of computational costs, we evaluate the number of hash computations during signature generation. The number of hash computations in $\text{HSIG}[\text{OTS}, \text{H}_{\text{cr}}, \text{PRF}]$ is $D(2^L(1 + \#\text{hash}_{\text{OTS.KGen}}) + 2^{L+1})$, since in the OTSs considered now, the one-time signature is determined during the key generation process. For OTS^{Lam} , the verification key and signature sizes are $|\text{ovk}| = |\sigma_{\text{OTS}}| = 8\lambda^2$ bits, and those of OTS^{LM} are $|\text{ovk}| = 2\lambda$ bits¹⁰ and $|\sigma_{\text{OTS}}| = 2\lambda(1 + 2\lambda/w + \lceil \log_2(2\lambda/w) \cdot (2^w - 1) \rceil / w)$ bits, where w is the so-called Winternitz parameter. Further, in Lamport signature, $\#\text{hash}_{\text{OTS.KGen}} = 4\lambda$; in OTS^{LM} , $\#\text{hash}_{\text{OTS.KGen}} = (2^w - 1)(2\lambda/w + \lceil \log_2(2\lambda/w) \cdot (2^w - 1) \rceil / w)$.

The concrete efficiency of the hash-based signature is summarized in Table 4. We set $L = D = 16$ to ensure the probability of using the same OTS secret key is less than 2^{-128} . The signature size with Lamport signature is about several hundred kilobytes, and the signature size with OTS^{LM} is several tens of kilobytes. Signature generation requires several hundred million hash computations.

6.3 Comparison

Finally, we compare our identification-based construction and hash-based signature schemes. Asymptotically, the bit length of the signature of both schemes is proportional to λ^2 ,¹¹ but the concrete values of them are different. The signature of the classical scheme $\text{SIG}[\text{ID}_{\text{Oka}}, \text{H}]$ is much shorter than the others. As for post-quantum schemes, $\text{SIG}[\text{ID}_{\text{OR}}[\text{ID}_{\text{CouSto}}], \text{H}]$ offers shorter signature than $\text{HSIG}[\text{OTS}, \text{H}_{\text{cr}}, \text{PRF}]$. Regarding

⁹ The signature size of Pan and Wagner's scheme can be reduced from $2\lambda(2|\mathcal{E}| + |\mathcal{G}|)$ to $2\lambda(|\mathcal{G}| + 1)$ by applying the optimization in Section 3.2.

¹⁰ The verification key of OTS^{LM} consists of one hash value, the signer identity I , and the instance identity *num*. But I and *num* can be substituted by *svk* and $r_1 \parallel \dots \parallel r_D$ when used in $\text{HSIG}[\text{OTS}, \text{H}_{\text{cr}}, \text{PRF}]$. So, we omit them in our analysis.

¹¹ The signature of both schemes consists of $\mathcal{O}(\lambda)$ elements of λ bits.

computational costs, the number of hash computations in our construction is significantly smaller than that of $\text{HSIG}[\text{OTS}, \text{H}_{\text{cr}}, \text{PRF}]$. However, we cannot conclude that our constructions are better in terms of computational costs because they require heavy group operations in addition to lightweight hash computations.

Acknowledgments. We thank anonymous reviewers for their valuable comments and suggestions. They informed us of the works [KRW24, Sch22] that pointed out flaws in the randomized Fischlin transformation. This work was partially supported by JST CREST JPMJCR22M1, Japan.

References

- ADMP20. Navid Alapati, Luca De Feo, Hart Montgomery, and Sikhar Patranabis. Cryptographic group actions and applications. In Shiho Moriai and Huaxiong Wang, editors, *ASIACRYPT 2020, Part II*, volume 12492 of *LNCS*, pages 411–439. Springer, Cham, December 2020.
- Bad14. Christoph Bader. Efficient signatures with tight real world security in the random-oracle model. In Dimitris Gritzalis, Aggelos Kiayias, and Ioannis G. Askoxylakis, editors, *CANS 14*, volume 8813 of *LNCS*, pages 370–383. Springer, Cham, October 2014.
- BDK⁺07. Johannes Buchmann, Erik Dahmen, Elena Klintsevich, Katsuyuki Okeya, and Camille Vuillaume. Merkle signatures with virtually unlimited signature capacity. In Jonathan Katz and Moti Yung, editors, *ACNS 07 International Conference on Applied Cryptography and Network Security*, volume 4521 of *LNCS*, pages 31–45. Springer, Berlin, Heidelberg, June 2007.
- BHJ⁺15. Christoph Bader, Dennis Hofheinz, Tibor Jager, Eike Kiltz, and Yong Li. Tightly-secure authenticated key exchange. In Yevgeniy Dodis and Jesper Buus Nielsen, editors, *TCC 2015, Part I*, volume 9014 of *LNCS*, pages 629–658. Springer, Berlin, Heidelberg, March 2015.
- BKV19. Ward Beullens, Thorsten Kleinjung, and Frederik Vercauteren. CSI-FiSh: Efficient isogeny based signatures through class group computations. In Steven D. Galbraith and Shiho Moriai, editors, *ASIACRYPT 2019, Part I*, volume 11921 of *LNCS*, pages 227–247. Springer, Cham, December 2019.
- CDS94. Ronald Cramer, Ivan Damgård, and Berry Schoenmakers. Proofs of partial knowledge and simplified design of witness hiding protocols. In Yvo Desmedt, editor, *CRYPTO’94*, volume 839 of *LNCS*, pages 174–187. Springer, Berlin, Heidelberg, August 1994.
- CL24. Yi-Hsiu Chen and Yehuda Lindell. Optimizing and implementing fischlin’s transform for UC-secure zero knowledge. *IACR Communications in Cryptology*, 1(2), 2024.
- Cou06. Jean-Marc Couveignes. Hard homogeneous spaces. Cryptology ePrint Archive, Report 2006/291, 2006.
- DFMS22. Jelle Don, Serge Fehr, Christian Majenz, and Christian Schaffner. Online-extractability in the quantum random-oracle model. In Orr Dunkelman and Stefan Dziembowski, editors, *EUROCRYPT 2022, Part III*, volume 13277 of *LNCS*, pages 677–706. Springer, Cham, May / June 2022.
- DGJL21. Denis Diemert, Kai Gellert, Tibor Jager, and Lin Lyu. More efficient digital signatures with tight multi-user security. In Juan Garay, editor, *PKC 2021, Part II*, volume 12711 of *LNCS*, pages 1–31. Springer, Cham, May 2021.
- Eat17. Edward Eaton. Leighton-Micali hash-based signatures in the quantum random-oracle model. In Carlisle Adams and Jan Camenisch, editors, *SAC 2017*, volume 10719 of *LNCS*, pages 263–280. Springer, Cham, August 2017.
- FF02. Marc Fischlin and Roger Fischlin. The representation problem based on factoring. In Preneel [Pre02], pages 96–113.
- Fis02. Marc Fischlin. On the impossibility of constructing non-interactive statistically-secret protocols from any trapdoor one-way function. In Preneel [Pre02], pages 79–95.
- Fis05. Marc Fischlin. Communication-efficient non-interactive proofs of knowledge with online extractors. In Victor Shoup, editor, *CRYPTO 2005*, volume 3621 of *LNCS*, pages 152–168. Springer, Berlin, Heidelberg, August 2005.
- FS87. Amos Fiat and Adi Shamir. How to prove yourself: Practical solutions to identification and signature problems. In Andrew M. Odlyzko, editor, *CRYPTO’86*, volume 263 of *LNCS*, pages 186–194. Springer, Berlin, Heidelberg, August 1987.
- GJ18. Kristian Gjøsteen and Tibor Jager. Practical and tightly-secure digital signatures and authenticated key exchange. In Hovav Shacham and Alexandra Boldyreva, editors, *CRYPTO 2018, Part II*, volume 10992 of *LNCS*, pages 95–125. Springer, Cham, August 2018.

- GMR88. Shafi Goldwasser, Silvio Micali, and Ronald L. Rivest. A digital signature scheme secure against adaptive chosen-message attacks. *SIAM Journal on Computing*, 17(2):281–308, April 1988.
- HBD⁺22. Andreas Hülsing, Daniel J. Bernstein, Christoph Dobraunig, Maria Eichlseder, Scott Fluhrer, Stefan-Lukas Gazdag, Panos Kampanakis, Stefan Kölbl, Tanja Lange, Martin M. Lauridsen, Florian Mendel, Ruben Niederhagen, Christian Rechberger, Joost Rijneveld, Peter Schwabe, Jean-Philippe Aumasson, Bas Westerbaan, and Ward Beullens. SPHINCS⁺. Technical report, National Institute of Standards and Technology, 2022. available at <https://csrc.nist.gov/Projects/post-quantum-cryptography/selected-algorithms-2022>.
- HJK⁺21. Shuai Han, Tibor Jager, Eike Kiltz, Shengli Liu, Jiaxin Pan, Doreen Riepel, and Sven Schäge. Authenticated key exchange and signatures with tight security in the standard model. In Tal Malkin and Chris Peikert, editors, *CRYPTO 2021, Part IV*, volume 12828 of *LNCS*, pages 670–700, Virtual Event, August 2021. Springer, Cham.
- Kat16. Jonathan Katz. Analysis of a proposed hash-based signature standard. In Lidong Chen, David McGrew, and Chris Mitchell, editors, *Security Standardisation Research*, pages 261–273, Cham, 2016. Springer International Publishing.
- KMP16. Eike Kiltz, Daniel Masny, and Jiaxin Pan. Optimal security proofs for signatures from identification schemes. In Matthew Robshaw and Jonathan Katz, editors, *CRYPTO 2016, Part II*, volume 9815 of *LNCS*, pages 33–61. Springer, Berlin, Heidelberg, August 2016.
- KRW24. Michael Klooß, Michael Reichle, and Benedikt Wagner. Practical blind signatures in pairing-free groups. In Kai-Min Chung and Yu Sasaki, editors, *Advances in Cryptology – ASIACRYPT 2024, Part I*, volume 15484 of *Lecture Notes in Computer Science*, pages 363–395, Kolkata, India, December 9–13, 2024. Springer, Singapore, Singapore.
- Ks22. Yashvanth Kondi and abhi shelat. Improved straight-line extraction in the random oracle model with applications to signature aggregation. In Shweta Agrawal and Dongdai Lin, editors, *ASIACRYPT 2022, Part II*, volume 13792 of *LNCS*, pages 279–309. Springer, Cham, December 2022.
- LPLL20. Youngkyung Lee, Jong Hwan Park, Kwangsu Lee, and Dong Hoon Lee. Tight security for the generic construction of identity-based signature (in the multi-instance setting). *Theoretical Computer Science*, 847:122–133, 2020.
- MCF19. David McGrew, Michael Curcio, and Scott Fluhrer. Leighton-Micali Hash-Based Signatures. RFC 8554, April 2019.
- Mer79. Ralph Charles Merkle. *Secrecy, authentication, and public key systems*. PhD thesis, Stanford, CA, USA, 1979. AAI8001972.
- Mer90. Ralph C. Merkle. A certified digital signature. In Gilles Brassard, editor, *CRYPTO’89*, volume 435 of *LNCS*, pages 218–238. Springer, New York, August 1990.
- Oka93. Tatsuoaki Okamoto. Provably secure and practical identification schemes and corresponding signature schemes. In Ernest F. Brickell, editor, *CRYPTO’92*, volume 740 of *LNCS*, pages 31–53. Springer, Berlin, Heidelberg, August 1993.
- Pas03. Rafael Pass. On deniability in the common reference string and random oracle model. In Dan Boneh, editor, *CRYPTO 2003*, volume 2729 of *LNCS*, pages 316–337. Springer, Berlin, Heidelberg, August 2003.
- PQR21. Jiaxin Pan, Chen Qian, and Magnus Ringerud. Signed Diffie-Hellman key exchange with tight security. In Kenneth G. Paterson, editor, *CT-RSA 2021*, volume 12704 of *LNCS*, pages 201–226. Springer, Cham, May 2021.
- PQR22. Jiaxin Pan, Chen Qian, and Magnus Ringerud. Signed (group) Diffie-Hellman key exchange with tight security. *Journal of Cryptology*, 35(4):26, October 2022.
- Pre02. Bart Preneel, editor. *CT-RSA 2002*, volume 2271 of *LNCS*. Springer, Berlin, Heidelberg, February 2002.
- PW22. Jiaxin Pan and Benedikt Wagner. Lattice-based signatures with tight adaptive corruptions and more. In Goichiro Hanaoka, Junji Shikata, and Yohei Watanabe, editors, *PKC 2022, Part II*, volume 13178 of *LNCS*, pages 347–378. Springer, Cham, March 2022.
- Sch22. Tim Scheurer. Universally composable verifiable random oracles. Master’s thesis, Karlsruher Institut für Technologie (KIT), 2022.
- Sto12. Anton Stolbunov. *Cryptographic Schemes Based on Isogenies*. PhD thesis, Norwegian University of Science and Technology, 2012.
- WLG⁺19. Ge Wu, Jian-Chang Lai, Fu-Chun Guo, Willy Susilo, and Fu-Tai Zhang. Tightly secure public-key cryptographic schemes from one-more assumptions. *Journal of Computer Science and Technology*, 34:1366–1379, 2019.
- YOH24. Hirofumi Yoshioka, Wakaha Ogata, and Keitaro Hashimoto. Towards a tightly secure signature in multi-user setting with corruptions based on search assumptions. Cryptology ePrint Archive, Report 2024/1286, 2024.

A Proof of Theorem 2

This section provides the full proof of Theorem 2.

Proof. Let $\mathcal{A}$ be a PPT adversary against the MU-SUF-CMA-C security of $\text{SIG}[\text{ID}, \text{H}]$ and $\mathcal{C}$ be a challenger managing the security game. To prove the theorem, we consider the following sequence of games.

Game₀. This is the original MU-SUF-CMA-C game. By definition, we have

$$\Pr[\text{Game}_0 \Rightarrow 1] = \text{Adv}_{\mathcal{A}, \text{SIG}[\text{ID}, \text{H}]}^{\text{MU-SUF-CMA-C}}(\lambda).$$

In the following, for a query $(\text{pk}, m, \text{com}, j, \text{ch}, \text{resp})$ to the RO, we call $(\text{pk}, m, \text{com})$ a prefix of the query. If $V(\text{pk}, \text{com}_j, \text{ch}, \text{resp}) = 1$, the query is called a “valid transcript”, otherwise, it is called an “invalid transcript.”

Game₁. In this game, $\mathcal{C}$ simulates RO using three lists $L_{\text{RO}}^{\text{valid}}, L_{\text{RO}}^{\text{invalid}}, L_{\text{RO}}^{\text{ignore}}$, and simulates $\text{OSign}(i, m)$ as in Figure 8 (without the boxed line). In the simulation of OSign , $\mathcal{C}$ computes the responses and the hash values for all ch instead of computing them one by one until it finds a challenge whose corresponding hash value is 0^γ .

On the other hand, $\mathcal{C}$ simulates $\text{RO}(\text{pk}, m, \text{com}, j, \text{ch}, \text{resp})$ as follows: $\mathcal{C}$ first checks if the same query has already been issued based on $L_{\text{RO}} := L_{\text{RO}}^{\text{valid}} \cup L_{\text{RO}}^{\text{invalid}} \cup L_{\text{RO}}^{\text{ignore}}$. If so, $\mathcal{C}$ returns the consistent value. Otherwise, chooses $h \leftarrow \{0, 1\}^\gamma$, adds a query-answer tuple $(\text{pk}, m, \text{com}, j, \text{ch}, \text{resp}, h)$ to one of three lists according to the following conditions, and returns h as the hash value.

- When the query is issued by $\mathcal{A}$:
 - if it is a valid transcript, the tuple $(\text{pk}, m, \text{com}, j, \text{ch}, \text{resp}, h)$ is added to $L_{\text{RO}}^{\text{valid}}$.
 - if it is an invalid transcript, the tuple is added to $L_{\text{RO}}^{\text{invalid}}$.
- When the query is issued internally in the simulation of $\text{OSign}(i, m)$:
 - if com and $(\text{com}_j, j, \text{ch}, \text{resp})$ are used in the signature, then the tuple $(\text{pk}_i, m, \text{com}, j, \text{ch}, \text{resp}, h)$ is added to $L_{\text{RO}}^{\text{valid}}$,
 - if com is used in the signature, but $(\text{com}_j, j, \text{ch}, \text{resp})$ is not, then the tuple is added to $L_{\text{RO}}^{\text{ignore}}$,
 - if com is not used in the signature, the signer must have asked 2^t queries $(\text{pfx}, j, \text{ch}', \text{resp}', h')$ with the same (pfx, j) . From these queries, one query and its answer, chosen at random in Line 16, is added to $L_{\text{RO}}^{\text{valid}}$, and the rest are added to $L_{\text{RO}}^{\text{ignore}}$ ¹².
- When $\mathcal{C}$ verifies the forged signature, we consider $\mathcal{A}$ makes hash queries $(\text{pk}_{i^*}, m^*, \text{com}^*, j, \text{ch}_j^*, \text{resp}_j^*)$ for all j . According to these queries, $L_{\text{RO}}^{\text{valid}}$ or $L_{\text{RO}}^{\text{invalid}}$ are updated depending on $V(\text{pk}_{i^*}, \text{com}_j^*, \text{ch}_j^*, \text{resp}_j^*) = 1$ or not.

The function `UpdateLists` in Figure 9 specifies the concrete way to update $L_{\text{RO}}^{\text{valid}}$ and $L_{\text{RO}}^{\text{ignore}}$. Note that the way $\text{ch}_{\tau, j}$ is determined depends on $\tau = \hat{\tau}$ or not. From this process, for any $(\text{pk}_i, m, \text{com}, j)$, at most one tuple $(\text{pk}_i, m, \text{com}, j, *, *, *)$ is added to $L_{\text{RO}}^{\text{valid}}$ in the process of OSign .

The modifications in how the challenges adopted in the signature are determined do not change the probability distribution of the signature. Further, the change in handling the random oracle is just conceptual. Therefore, we have

$$\Pr[\text{Game}_1 \Rightarrow 1] = \Pr[\text{Game}_0 \Rightarrow 1].$$

Game₂. In this game, the simulation of OSign is further changed as follows. After setting pfx_τ in Line 7 of Figure 8, $\mathcal{C}$ browses the list $L_{\text{RO}}^{\text{valid}}$ to find a tuple whose prefix is pfx_τ . If such a tuple exists, the sign oracle immediately returns $\perp$. Otherwise, $\mathcal{C}$ proceeds the signing process as in Game₁.

Because honestly generated commitments com has $\rho\kappa$ bits min-entropy and the number of varieties of com in $L_{\text{RO}}^{\text{valid}}$ is at most $Q_{\text{RO}} + T \cdot Q_{\text{SIG}}$, the difference between Game₁ and Game₂ is upper bounded by $T \cdot Q_{\text{SIG}} \times (Q_{\text{RO}} + T \cdot Q_{\text{SIG}})/2^{\rho\kappa}$. Therefore,

$$|\Pr[\text{Game}_2 \Rightarrow 1] - \Pr[\text{Game}_1 \Rightarrow 1]| \leq T \cdot Q_{\text{SIG}}(Q_{\text{RO}} + T \cdot Q_{\text{SIG}})2^{-\rho\kappa}.$$

¹² Here, we need to keep one query-answer pair in $L_{\text{RO}}^{\text{valid}}$ to simulate the random oracle consistently in the following games; instead of responding to discarded queries with a random value (that can be 0^t and then differs from the original game), the simulator utilizes such queries to extract a secret key.

```

OSign( $i, m$ )
1 : if  $i \in L_{\text{corr}}$  then return  $\perp$ 
2 : foreach  $\tau \in [T]$  do
3 :   foreach  $j \in [\rho]$  do
4 :      $(\text{com}_{\tau,j}, \text{st}_{\tau,j}) \leftarrow P_1(\text{par}, \text{pk}_i, \text{sk}_i)$ 
5 :   endfor
6 :    $\text{com}_{\tau} := (\text{com}_{\tau,1}, \dots, \text{com}_{\tau,\rho})$ 
7 :    $\text{pfx}_{\tau} := (\text{pk}_i, m, \text{com}_{\tau})$ 
8 :   if  $(\text{pfx}_{\tau}, *, *, *) \in L_{\text{RO}}^{\text{valid}}$  then return  $\perp$ 
9 :   // Check if  $\text{pfx}_{\tau}$  was not generated before
10 :  foreach  $j \in [\rho]$  do
11 :    foreach  $\text{ch} \in \{0, 1\}^t$  do
12 :       $\text{resp}_{\tau,j,\text{ch}} \leftarrow P_2(\text{ch}, \text{st}_{\tau,j})$ 
13 :       $h_{\tau,j,\text{ch}} \leftarrow H(\text{pfx}_{\tau}, j, \text{ch}, \text{resp}_{\tau,j,\text{ch}})$ 
14 :    endfor
15 :    if  $\{\text{ch} \mid h_{\tau,j,\text{ch}} = 0^\gamma\} = \emptyset$ 
16 :       $\text{ch}_{\tau,j} \leftarrow \mathbb{S} \{0, 1\}^t$ 
17 :    else
18 :       $\text{ch}_{\tau,j} \leftarrow \mathbb{S} \{\text{ch} \mid h_{\tau,j,\text{ch}} = 0^\gamma\}$ 
19 :     $\text{UpdateLists}(\text{pfx}_{\tau}, j, \{\text{resp}_{\tau,j,\text{ch}}, h_{\tau,j,\text{ch}}\}_{\text{ch} \in \{0,1\}^t}, \text{ch}_{\tau,j})$ 
20 :  endfor
21 :  if  $\forall j : h_{\tau,j,\text{ch}_{\tau,j}} = 0^\gamma$  then break
22 :  if  $\tau = T$  then  $\sigma := \perp$ 
23 : endfor
24 :  $\hat{\tau} := \tau$ 
25 :  $\sigma := (\text{com}_{\hat{\tau},j}, \text{ch}_{\hat{\tau},j}, \text{resp}_{\hat{\tau},j,\text{ch}_{\hat{\tau},j}})_{j \in [\rho]}$ 
26 :  $L_{\text{sig}} := L_{\text{sig}} \cup \{(i, m, \sigma)\}$ 
27 : return  $\sigma$ 

```

Fig. 8: The sign oracle in Game_1 (without boxed line) and Game_2 (with boxed line).

```

UpdateLists( $\text{pfx}_{\tau}, j, \{\text{resp}_{\tau,j,\text{ch}}, h_{\tau,j,\text{ch}}\}_{\text{ch} \in \{0,1\}^t}, \text{ch}_{\tau,j}$ )
1 : foreach  $\text{ch} \in \{0, 1\}^t$ 
2 :   if  $\text{ch} = \text{ch}_{\tau,j}$  then
3 :     add  $(\text{pfx}_{\tau}, j, \text{ch}, \text{resp}_{\tau,j,\text{ch}}, h_{\tau,j,\text{ch}})$  to  $L_{\text{RO}}^{\text{valid}}$ 
4 :   else
5 :     add  $(\text{pfx}_{\tau}, j, \text{ch}, \text{resp}_{\tau,j,\text{ch}}, h_{\tau,j,\text{ch}})$  to  $L_{\text{RO}}^{\text{ignore}}$ 
6 :   endif
7 : endfor

```

Fig. 9: Function UpdateLists used in OSign .

Game₃. In this game, we introduce a flag F_{find} which is initialized as $F_{\text{find}} := \text{false}$. The simulation of RO is changed as follows: When $(\text{pk}_i, m, \text{cōm}, j, \text{ch}, \text{resp}, h)$ is added to $L_{\text{RO}}^{\text{valid}}$ as the response of $\mathcal{A}$'s query and $i \notin L_{\text{corr}}$ and $F_{\text{find}} = \text{false}$ hold, $\mathcal{C}$ browses the list $L_{\text{RO}}^{\text{valid}}$ and finds a tuple $(\text{pk}_i, m, \text{cōm}, j, \text{ch}', \text{resp}', h')$ such that $(\text{ch}', \text{resp}') \neq (\text{ch}, \text{resp})$. If such a tuple exists, $\mathcal{C}$ sets

$$\begin{aligned} \text{pair} &:= (\text{pk}_i, \text{com}_j, \text{ch}, \text{resp}, \text{ch}', \text{resp}'), \\ F_{\text{find}} &:= \text{true}. \end{aligned}$$

Further, we add a condition " $F_{\text{find}} = \text{true}$ " to the requirements that the game outputs 1.

Because $\mathcal{A}$ cannot see F_{find} and pair , the change of RO simulation does not affect $\mathcal{A}$'s view. Thus, $|\Pr[\text{Game}_3 \Rightarrow 1] - \Pr[\text{Game}_2 \Rightarrow 1]|$ is bounded by the probability that $\mathcal{A}$'s final outputs is valid and $F_{\text{find}} = \text{false}$ at the end of **Game₂**. Let **Lucky** be the event that both of these two conditions hold. We will evaluate $\Pr[\text{Lucky}]$ in the following three cases. Let $\sigma^* = (\text{com}_j^*, \text{ch}_j^*, \text{resp}_j^*)_{j \in [\rho]}$ be $\mathcal{A}$'s forged signature, and $\text{pfx}^* := (\text{pk}_{i^*}, m^*, \text{cōm}^*)$.

- (1) pfx^* is equal to one of pfxs generated in **OSign** and the oracle returned a signature including it.
- (2) pfx^* is equal to one of pfxs generated in **OSign** but the oracle returned a signature that does not include pfx^* .
- (3) pfx^* was not used in **OSign**.

Due to the condition that F_{find} is set to **true** and the fact that $L_{\text{RO}}^{\text{valid}}$ includes all transcripts in the signatures returned from **OSign** and one transcript per each j whose prefix was considered in **OSign** but discarded, in cases (1) and (2), query-answer tuples $(\text{pfx}^*, j, \text{ch}_j^*, \text{resp}_j^*, h_j)$ for all j should have been added in the process of **OSign**. In case (1), if all transcripts in σ^* are valid and $F_{\text{find}} = \text{false}$, then $(i^*, m^*, \sigma^*) \in L_{\text{SIG}}$ holds. Therefore, $\Pr[\text{Lucky}] = 0$. In case (2), if all transcripts in σ^* are valid and $F_{\text{find}} = \text{false}$, then there should exist j such that $h_j = H(\text{pfx}^*, j, \text{ch}_j^*, \text{resp}_j^*) \neq 0^\gamma$. Therefore, $\Pr[\text{Lucky}] = 0$.

On the other hand, in case (3), the event **Lucky** occurs only if $\mathcal{A}$ obtains valid transcripts $(\text{com}_j^*, \text{ch}_j, \text{resp}_j)$ such that $h_j = H(\text{pk}_{i^*}, m^*, \text{cōm}^*, j, \text{ch}_j, \text{resp}_j) = 0^\gamma$ for all $j \in [\rho]$ with a single hash computation. Since hash values are chosen independently and uniformly at random from $\{0, 1\}^\gamma$, the probability $h_j = 0^\gamma$ is $2^{-\gamma}$ for each $j \in [\rho]$. Thus, for a fixed $(\text{pk}_{i^*}, m^*, \text{cōm}^*)$, the probability $h_j = 0^\gamma$ for all $j \in [\rho]$ is bounded by $2^{-\rho\gamma}$. Since $\mathcal{A}$ issues Q_{RO} RO queries with any prefix $(\text{pk}, m, \text{cōm})$ and finally outputs $(\text{pk}_{i^*}, m^*, \text{cōm}^*)$ as a part of the forged signature, $\Pr[\text{Lucky}]$ is bounded by $(Q_{\text{RO}} + 1)/2^{\rho\gamma}$, and we have

$$|\Pr[\text{Game}_3 \Rightarrow 1] - \Pr[\text{Game}_2 \Rightarrow 1]| \leq (Q_{\text{RO}} + 1)2^{-\rho\gamma}.$$

In the following, let $\hat{i}$ be the index of the first element of $\text{pair} = (\text{pk}_{\hat{i}}, \dots)$.

Game₄. In this game, $\mathcal{C}$ computes the following when F_{find} is set to **true**.

$$\text{sk}^* \leftarrow \text{Ext}(\text{pair}) = \text{Ext}(\text{pk}_{\hat{i}}, \text{com}_j, \text{ch}_j, \text{resp}_j, \text{ch}'_j, \text{resp}'_j).$$

Further, we add a condition " $\text{VerKey}(\text{par}, \text{pk}_{\hat{i}}, \text{sk}^*) = 1$ " to the requirements that the game outputs 1. (Note that if $F_{\text{find}} = \text{false}$, the game outputs 0.)

$|\Pr[\text{Game}_4 \Rightarrow 1] - \Pr[\text{Game}_3 \Rightarrow 1]|$ is bounded by the probability that pair is assigned but the extractor Ext fails to find a valid secret key (i.e., the output sk^* of Ext does not satisfy $\text{VerKey}(\text{par}, \text{pk}_{\hat{i}}, \text{sk}^*) = 1$). We will evaluate this probability of failure. From the condition of setting F_{find} , $\text{pair} = (\text{pk}_{\hat{i}}, \text{com}_j, \text{ch}_j, \text{resp}_j, \text{ch}'_j, \text{resp}'_j)$ satisfies

$$\text{V}(\text{pk}_{\hat{i}}, \text{com}_j, \text{ch}_j, \text{resp}_j) = \text{V}(\text{pk}_{\hat{i}}, \text{com}_j, \text{ch}'_j, \text{resp}'_j) = 1$$

and $(\text{ch}_j, \text{resp}_j) \neq (\text{ch}'_j, \text{resp}'_j)$. Therefore, the probability of this failure is at most $\text{Adv}_{\mathcal{B}_1, \text{ID}}^{\text{SSS}}(\lambda)$ for some algorithm $\mathcal{B}_1$. Formally, we can construct an adversary $\mathcal{B}_1$ that breaks the strong special soundness by using $\mathcal{A}$ as follows: Upon receiving a parameter par , $\mathcal{B}_1$ simulates **Game₄** against $\mathcal{A}$. If $\mathcal{B}_1$ obtains pair , it outputs pair . We can see $\mathcal{B}_1$ breaks the strong special soundness when Ext fails extraction. Thus, we have

$$|\Pr[\text{Game}_4 \Rightarrow 1] - \Pr[\text{Game}_3 \Rightarrow 1]| \leq \text{Adv}_{\mathcal{B}_1, \text{ID}}^{\text{SSS}}(\lambda).$$

We also evaluate the running time of $\mathcal{B}_1$. It executes $\mathcal{A}$ once and answers oracle queries from $\mathcal{A}$. Since each oracle query can be answered in the time of $\text{poly}(\lambda)$, the running time of $\mathcal{B}_1$ is $T_{\mathcal{B}_1} = T_{\mathcal{A}} + (Q_{\text{RO}} + Q_{\text{sig}} + Q_{\text{corr}}) \cdot \text{poly}(\lambda)$, where $Q_{\text{RO}}, Q_{\text{sig}}, Q_{\text{corr}}$ denote the maximum numbers of each oracle query $\mathcal{A}$ makes. Since $Q_{\text{RO}}, Q_{\text{sig}}, Q_{\text{corr}}$ are about $\mathcal{O}(T_{\mathcal{A}})$, we conclude that $T_{\mathcal{B}_1} = \mathcal{O}(T_{\mathcal{A}})$.

Game₅. This game outputs 1 if $(F_{\text{find}} = \text{true}) \wedge (\text{VerKey}(\text{par}, \text{pk}_i, \text{sk}^*) = 1)$ holds regardless of whether the forgery was successful or not.

Since the conditions $(F_{\text{find}} = \text{true})$ and $(\text{VerKey}(\text{par}, \text{pk}_i, \text{sk}^*) = 1)$ are already included in the requirements that **Game₄** outputs 1, clearly,

$$\Pr[\text{Game}_5 \Rightarrow 1] \geq \Pr[\text{Game}_4 \Rightarrow 1]$$

holds.

Game_{5'}. We add a condition $\text{sk}^* \neq \text{sk}_i$ to the requirements that the game outputs 1. Intuitively, from the perfect HVZK of ID, $\mathcal{A}$ cannot know which secret key $\mathcal{C}$ has among K secret keys corresponding pk_i . Therefore, we hope

$$\Pr[\text{Game}_{5'} \Rightarrow 1] \approx \frac{K-1}{K} \Pr[\text{Game}_5 \Rightarrow 1].$$

Actually, we can prove the following lemma.

Lemma 2. *If ID is perfect HVZK,*

$$\Pr[\text{Game}_{5'} \Rightarrow 1] = \frac{K-1}{K} \Pr[\text{Game}_5 \Rightarrow 1].$$

Before proving Lemma 2, we will upper-bound $\Pr[\text{Game}_{5'} \Rightarrow 1]$ and conclude the proof. We can construct an adversary $\mathcal{B}_2$ against the second key recovery resistance of ID using $\mathcal{A}$. $\mathcal{B}_2$ receives a public parameter par and key pairs of ID $\{(\text{pk}_i, \text{sk}_i)\}_{i \in [N]}$. It sets $(\text{svk}_i, \text{ssk}_i) := (\text{pk}_i, (\text{pk}_i, \text{sk}_i))$, initializes the lists $L_{\text{RO}}^{\text{valid}}, L_{\text{RO}}^{\text{invalid}}, L_{\text{RO}}^{\text{ignore}}, L_{\text{corr}}$ and L_{sig} and executes $\mathcal{A}$ on input $(\text{par}, \{\text{svk}_i\}_{i \in [N]})$ and answers $\mathcal{A}$'s oracle queries as in **Game_{5'}**. If $\mathcal{B}_2$ obtains one of pk_i 's valid secret keys $\text{sk}^* (\neq \text{sk}_i)$, $\mathcal{B}_2$ outputs (i, sk^*) .

We can verify that $\mathcal{B}_2$ perfectly simulates **Game_{5'}** against $\mathcal{A}$. In addition, due to the modifications we made in the previous games, **Game_{5'}** outputs 1 only if $F_{\text{find}} = \text{true}$ and sk^* is a valid secret key w.r.t. pk_i that is different from sk_i . Thus, $\mathcal{B}_2$ breaks the second key recovery resistance in the multi-user setting of ID. Therefore, we have

$$\Pr[\text{Game}_{5'} \Rightarrow 1] \leq \text{Adv}_{\mathcal{B}_2, \text{ID}}^{2^{\text{nd}} \text{KR}}(\lambda).$$

We also evaluate the running time of $\mathcal{B}_2$. It executes $\mathcal{A}$ once and answers oracle queries from $\mathcal{A}$. Since each oracle query can be answered in the time of $\text{poly}(\lambda)$, the running time of $\mathcal{B}_2$ is $T_{\mathcal{B}_2} = T_{\mathcal{A}} + (Q_{\text{RO}} + Q_{\text{sig}} + Q_{\text{corr}}) \cdot \text{poly}(\lambda)$, where $Q_{\text{RO}}, Q_{\text{sig}}, Q_{\text{corr}}$ denote the maximum number of each oracle query $\mathcal{A}$ made. Since $Q_{\text{RO}}, Q_{\text{sig}}, Q_{\text{corr}}$ are about $\mathcal{O}(T_{\mathcal{A}})$, we conclude that $T_{\mathcal{B}_2} = \mathcal{O}(T_{\mathcal{A}})$.

Combining everything, we have

$$\begin{aligned} \text{Adv}_{\mathcal{A}, \text{SIG}[\text{ID}, \text{H}]}^{\text{MU-SUF-CMA-C}}(\lambda) &\leq \text{Adv}_{\mathcal{B}_1, \text{ID}}^{\text{SSS}}(\lambda) + \frac{K}{K-1} \text{Adv}_{\mathcal{B}_2, \text{ID}}^{2^{\text{nd}} \text{KR}}(\lambda) \\ &\quad + \frac{(Q_{\text{RO}} + 1)}{2^{\rho\kappa}} + \frac{T \cdot Q_{\text{sig}}(Q_{\text{RO}} + T \cdot Q_{\text{sig}})}{2^{\rho\gamma}}. \end{aligned}$$

Our remaining task is proving Lemma 2. To do so, we use two game sequences started at **Game₅** and **Game_{5'}**. In the following, **Game_{X'}** is exactly the same as **Game_X** except the condition $\text{sk}^* \neq \text{sk}_i$ is added to the requirements that the game outputs 1.

Game₆ and Game_{6'}. First, we introduce RO programming, as Sim works well only if the RO is programmable. That is, Line 13 in Figure 8 is replaced with

$$h_{\tau, j, \text{ch}} \leftarrow_{\$} \{0, 1\}^{\gamma}$$

and the following line is inserted before Line 3 and Line 5 in Figure 9:

$$\boxed{H(\text{pfx}_\tau, j, \text{ch}, \text{resp}_{\tau,j,\text{ch}}) := h_{\tau,j,\text{ch}} \quad // \text{ program RO}}$$

This is a conceptual change. Thus, we have

$$\begin{aligned} \Pr[\text{Game}_6 \Rightarrow 1] &= \Pr[\text{Game}_5 \Rightarrow 1], \\ \Pr[\text{Game}_{6'} \Rightarrow 1] &= \Pr[\text{Game}_{5'} \Rightarrow 1]. \end{aligned}$$

Note that for each (pfx, j) , one of 2^t tuples $(\text{pfx}, j, \text{ch}, \text{resp}, h)$ is added to $L_{\text{RO}}^{\text{valid}}$ and the rest are added to $L_{\text{RO}}^{\text{ignore}}$. But all of them are programmed since both lists L^{valid} and L^{ignore} are used to answer the RO queries.

Game₇ and Game_{7'}. In this game, we program the RO for only one of 2^t hash values in the signing oracle, and the rest of them will be programmed only when the corresponding signer is corrupted. To do so, we prepare a new list L^{tmp} that keeps all query-answer pairs that may be programmed later. In particular, we delete the line before Line 5 that was added in Game₆, and replace Line 5 in Figure 9 with the next one.

$$\boxed{\text{add } (\text{pfx}_\tau, j, \text{ch}, \text{resp}_{\tau,j,\text{ch}}, h_{\tau,j,\text{ch}}) \text{ to } L^{\text{tmp}}}$$

Further, when $\text{OCorr}(i)$ is queried, for each tuple $(\text{pk}_i, m, \text{com}, j, \text{ch}, \text{resp}, h) \in L^{\text{tmp}}$, $\mathcal{C}$ programs the RO as $H(\text{pk}_i, m, \text{com}, j, \text{ch}, \text{resp}) := h$ and moves the tuple to $L_{\text{RO}}^{\text{ignore}}$.

$\mathcal{A}$'s views in Game₆ and Game₇ (resp. Game_{6'} and Game_{7'}) are exactly the same until $\mathcal{A}$ makes a query whose hash value is programmed in Game₆ (resp. Game_{6'}) but not in Game₇ (resp. Game_{7'}). Let $(\text{pfx}, j, \text{ch}', \text{resp}')$ be such a query. Then, there must exist a pair $(\text{ch}'', \text{resp}'')$ such that $(\text{pfx}, j, \text{ch}'', \text{resp}'')$ has been programmed and added to $L_{\text{RO}}^{\text{valid}}$, and $\text{ch}'' \neq \text{ch}'$. This means that when $\mathcal{A}$ issues such a query for the first time, F_{find} is set to true and pair and sk^* are assigned values. Therefore, the probability that $(F_{\text{find}} = \text{true}) \wedge (\text{VerKey}(\text{pk}_i, \text{sk}^*) = 1)$ holds is the same in Game₆ (resp. Game_{6'}) and Game₇ (resp. Game_{7'}). Thus, we have

$$\begin{aligned} \Pr[\text{Game}_7 \Rightarrow 1] &= \Pr[\text{Game}_6 \Rightarrow 1], \\ \Pr[\text{Game}_{7'} \Rightarrow 1] &= \Pr[\text{Game}_{6'} \Rightarrow 1]. \end{aligned}$$

Game₈ and Game_{8'}. In this game, $\text{ch}_{\tau,j}$ is decided before computing $\text{com}_{\tau,j}$ and $\text{resp}_{\tau,j,\text{ch}}$. In particular, the signing oracle is modified as shown in Figure 10.

Since all values $h_{\tau,j,\text{ch}}$ are randomly chosen independently of $\text{com}_{\tau,j}$ and $\text{resp}_{\tau,j,\text{ch}}$, this change of order is a conceptual change. Therefore, we have

$$\begin{aligned} \Pr[\text{Game}_8 \Rightarrow 1] &= \Pr[\text{Game}_7 \Rightarrow 1], \\ \Pr[\text{Game}_{8'} \Rightarrow 1] &= \Pr[\text{Game}_{7'} \Rightarrow 1]. \end{aligned}$$

Game₉ and Game_{9'}. In this game, in preparation for the final step, the response $\text{resp}_{\tau,j,\text{ch}}$ corresponding to the challenge $\text{ch} = \text{ch}_{\tau,j}$ is computed immediately after computing $\text{com}_{\tau,j}$, and computation of other $\text{resp}_{\tau,j,\text{ch}}$ ($\text{ch} \neq \text{ch}_{\tau,j}$) is postponed as late as possible. Specifically, OSign and UpdateLists are modified as in Figure 11 and Figure 12, respectively.

This change is completely conceptual. Thus, we have

$$\begin{aligned} \Pr[\text{Game}_9 \Rightarrow 1] &= \Pr[\text{Game}_8 \Rightarrow 1], \\ \Pr[\text{Game}_{9'} \Rightarrow 1] &= \Pr[\text{Game}_{8'} \Rightarrow 1]. \end{aligned}$$

Game₁₀ and Game_{10'}. In this game, the signing oracle no longer uses the prover's algorithms P_1 and P_2 to generate a transcript $(\text{com}_{\tau,j}, \text{ch}_{\tau,j}, \text{resp}_{\text{ch}_{\tau,j}})$ set in a signature, but simulates it using the simulator Sim . Specifically, Lines 11 to 12 in Figure 11 are replaced with the following.

```

OSign( $i, m$ )
1 : if  $i \in L_{\text{corr}}$  then return  $\perp$ 
2 : foreach  $\tau \in [T]$  do
3 :   foreach  $j \in [\rho]$  do
4 :     foreach  $\text{ch} \in \{0, 1\}^t$  do
5 :        $h_{\tau,j,\text{ch}} \leftarrow \mathbb{S} \{0, 1\}^\gamma$ 
6 :     endfor
7 :     if  $\{\text{ch} \in \{0, 1\}^t \mid h_{\tau,j,\text{ch}} = 0^\gamma\} = \emptyset$  then
8 :        $\text{ch}_{\tau,j} \leftarrow \mathbb{S} \{0, 1\}^t$ 
9 :     else
10 :       $\text{ch}_{\tau,j} \leftarrow \mathbb{S} \{\text{ch} \in \{0, 1\}^t \mid h_{\tau,j,\text{ch}} = 0^\gamma\}$ 
11 :       $(\text{com}_{\tau,j}, \text{st}_{\tau,j}) \leftarrow P_1(\text{par}, \text{pk}_i, \text{sk}_i)$ 
12 :    endfor
13 :     $\text{com}_\tau := (\text{com}_{\tau,1}, \dots, \text{com}_{\tau,\rho})$ 
14 :     $\text{pfx}_\tau := (\text{pk}_i, m, \text{com}_\tau)$ 
15 :    if  $(\text{pfx}_\tau, *, *, *) \in L_{\text{RO}}^{\text{valid}}$  then return  $\perp$ 
16 :    foreach  $j \in [\rho]$  do
17 :      foreach  $\text{ch} \in \{0, 1\}^t$  do
18 :         $\text{resp}_{\tau,j,\text{ch}} \leftarrow P_2(\text{ch}, \text{st}_{\tau,j})$ 
19 :         $\text{UpdateLists}(\text{pfx}_\tau, j, \{\text{resp}_{\tau,j,\text{ch}}, h_{\tau,j,\text{ch}}\}_{\text{ch}}, \text{ch}_{\tau,j})$ 
20 :      endfor
21 :      if  $\forall j : h_{\tau,j,\text{ch}_{\tau,j}} = 0^\gamma$  then break
22 :      if  $\tau = T$  then  $\sigma := \perp$ 
23 :    endfor
24 :     $\hat{\tau} := \tau$ 
25 :     $\sigma := (\text{com}_{\hat{\tau},j}, \text{ch}_{\hat{\tau},j}, \text{resp}_{\hat{\tau},j,\text{ch}_{\hat{\tau},j}})_{j \in [\rho]}$ 
26 :     $L_{\text{sig}} := L_{\text{sig}} \cup \{(i, m, \sigma)\}$ 
27 :    return  $\sigma$ 

```

Fig. 10: The sign oracle in Game_8 and $\text{Game}_{8'}$.

$$(\text{com}_{\tau,j}, \text{resp}_{\tau,j,\text{ch}_{\tau,j}}) \leftarrow \text{Sim}(\text{pk}_i, \text{ch}_{\tau,j})$$

In contrast, $2^t - 1$ transcripts not set in a signature are computed based on the signing key. Note that Sim can properly simulate a transcript for only one challenge, but transcripts for other challenges that are consistent with the simulated one can be calculated using the signing key. More precisely, the next line is inserted

$$\{\text{resp}'_{\text{ch}}\}_{\text{ch} \in \{0,1\}^t \setminus \{\text{ch}_{\tau,j}\}} \leftarrow \text{ReSim}(\text{pk}_i, \text{sk}_i, \text{com}_{\tau,j}, \text{ch}_{\tau,j}, \text{resp}_{\tau,j,\text{ch}_{\tau,j}})$$

at the beginning of Figure 12, and Line 6 is replaced with

$$\text{resp}_{\tau,j,\text{ch}} := \text{resp}'_{\text{ch}}$$

where a function ReSim is defined as follows.

```

OSign( $i, m$ )
1 : if  $i \in L_{\text{corr}}$  then return  $\perp$ 
2 : foreach  $\tau \in [T]$  do
3 :   foreach  $j \in [\rho]$  do
4 :     foreach  $\text{ch} \in \{0, 1\}^t$  do
5 :        $h_{\tau,j,\text{ch}} \leftarrow \mathbb{S} \{0, 1\}^\gamma$ 
6 :     endfor
7 :     if  $\{\text{ch} \in \{0, 1\}^t \mid h_{\tau,j,\text{ch}} = 0^\gamma\} = \emptyset$  then
8 :        $\text{ch}_{\tau,j} \leftarrow \mathbb{S} \{0, 1\}^t$ 
9 :     else
10 :       $\text{ch}_{\tau,j} \leftarrow \mathbb{S} \{\text{ch} \in \{0, 1\}^t \mid h_{\tau,j,\text{ch}} = 0^\gamma\}$ 
11 :       $(\text{com}_{\tau,j}, \text{st}_{\tau,j}) \leftarrow P_1(\text{par}, \text{pk}_i, \text{sk}_i)$ 
12 :       $\text{resp}_{\tau,j,\text{ch}_{\tau,j}} \leftarrow P_2(\text{ch}_{\tau,j}, \text{st}_{\tau,j})$ 
13 :    endfor
14 :     $\text{com}_\tau := (\text{com}_{\tau,1}, \dots, \text{com}_{\tau,\rho})$ 
15 :     $\text{pfx}_\tau := (\text{pk}_i, m, \text{com}_\tau)$ 
16 :    if  $(\text{pfx}_\tau, *, *, *) \in L_{\text{RO}}^{\text{valid}}$  then return  $\perp$ 
17 :    foreach  $j \in [\rho]$  do
18 :       $\text{UpdateLists}(\text{pfx}_\tau, j, \{\text{resp}_{\tau,j,\text{ch}}, h_{\tau,j,\text{ch}}\}_{\text{ch}}, \text{ch}_{\tau,j})$ 
19 :    endfor
20 :    if  $\forall j : h_{\tau,j,\text{ch}_{\tau,j}} = 0^\gamma$  then break
21 :    if  $\tau = T$  then  $\sigma := \perp$ 
22 :  endfor
23 :   $\hat{\tau} := \tau$ 
24 :   $\sigma := (\text{com}_{\hat{\tau},j}, \text{ch}_{\hat{\tau},j}, \text{resp}_{\hat{\tau},j,\text{ch}_{\hat{\tau},j}})_{j \in [\rho]}$ 
25 :   $L_{\text{sig}} := L_{\text{sig}} \cup \{(i, m, \sigma)\}$ 
26 : return  $\sigma$ 

```

Fig. 11: The sign oracle in Game_9 and $\text{Game}_{9'}$.

```

UpdateLists( $\text{pfx}_\tau, j, \{\text{resp}_{\tau,j,\text{ch}}, h_{\tau,j,\text{ch}}\}_{\text{ch}}, \text{ch}_{\tau,j}$ )
1 : foreach  $\text{ch} \in \{0, 1\}^t$ 
2 :   if  $\text{ch} = \text{ch}_{\tau,j}$  then
3 :      $H(\text{pfx}_\tau, j, \text{ch}, \text{resp}_{\tau,j,\text{ch}}) := h_{\tau,j,\text{ch}}$  // program RO
4 :     add  $(\text{pfx}_\tau, j, \text{ch}, \text{resp}_{\tau,j,\text{ch}}, h_{\tau,j,\text{ch}})$  to  $L_{\text{RO}}^{\text{valid}}$ 
5 :   else
6 :      $\text{resp}_{\tau,j,\text{ch}} \leftarrow P_2(\text{ch}, \text{st}_{\tau,j})$ 
7 :     add  $(\text{pfx}_\tau, j, \text{ch}, \text{resp}_{\tau,j,\text{ch}}, h_{\tau,j,\text{ch}})$  to  $L^{\text{tmp}}$ 
8 :   endif
9 : endfor

```

Fig. 12: Function UpdateLists used in Game_9 .

- ReSim takes a valid key pair $(pk, sk) \in \text{IGen}(\text{par})$ and a valid transcript $(\text{com}, \text{ch}, \text{resp}) \in \text{Tran}(pk, sk, \text{ch})$ as input,
- chooses $r \leftarrow \$ \{r \mid (\text{com}, \text{ch}, \text{resp}) \leftarrow \text{Tran}(pk, sk, \text{ch}; r)\}$, where r is the random coins used to compute P_1 in Tran algorithm,
- regenerates $(\text{com}, \text{st}) \leftarrow P_1(pk, sk; r)$,
- computes $\text{resp}'_{\text{ch}'} \leftarrow P_2(\text{st}, \text{ch}')$ for all $\text{ch}' \in \{0, 1\}^t \setminus \{\text{ch}\}$, and
- outputs a list $(\text{resp}'_{\text{ch}'})_{\text{ch}' \in \{0, 1\}^t \setminus \{\text{ch}\}}$.

If the identification scheme is perfect HVZK, there exists r such that $(\text{com}, \text{ch}, \text{resp}) \leftarrow \text{Tran}(pk, sk, \text{ch}; r)$ for any transcript $(\text{com}, \text{ch}, \text{resp})$ simulated by Sim and for any valid sk . Thus, ReSim can generate the response resp' that would be computed by an honest prover having the specified secret key sk as the response to the different challenge ch' .

If the identification scheme is perfect HVZK, $\mathcal{A}$'s view does not change from the definition of ReSim. Thus,

$$\begin{aligned} \Pr[\text{Game}_{10} \Rightarrow 1] &= \Pr[\text{Game}_9 \Rightarrow 1], \\ \Pr[\text{Game}_{10'} \Rightarrow 1] &= \Pr[\text{Game}_{9'} \Rightarrow 1]. \end{aligned}$$

In Game_{10} and $\text{Game}_{10'}$, the value of sk_i is completely hidden from $\mathcal{A}$ until user i is corrupted, since sk_i is used only in `UpdateLists` to compute resp' , and the tuple including resp' is added to L^{tmp} which is perfectly hidden to $\mathcal{A}$ before the corruption of i . From the definition of $\hat{i}$, by the time F_{find} is set true and *pair* is assigned, $\hat{i}$ has not been corrupted, and thus (even unbounded) $\mathcal{A}$ has no information about $sk_{\hat{i}}$. Therefore, we have

$$\begin{aligned} \Pr[\text{Game}_{10'} \Rightarrow 1] &= \Pr[F_{\text{find}} = \text{true} \wedge \text{VerKey}(\text{par}, pk_{\hat{i}}, sk^*) = 1 \wedge sk_{\hat{i}} \neq sk^*] \\ &= \Pr[F_{\text{find}} = \text{true} \wedge \text{VerKey}(\text{par}, pk_{\hat{i}}, sk^*) = 1] \\ &\quad \times \Pr[sk_{\hat{i}} \neq sk^* \mid F_{\text{find}} = \text{true} \wedge \text{VerKey}(\text{par}, pk_{\hat{i}}, sk^*) = 1] \\ &= \Pr[\text{Game}_{10} \Rightarrow 1] \times \frac{K-1}{K}. \end{aligned}$$

This completes the proof of Lemma 2. □

B Cryptographic Group Action

Here, we recall cryptographic group action.

Definition 14 (Group Action). A group $\mathcal{G}$ is said to act on a set $\mathcal{E}$ if there is a map $\star : \mathcal{G} \times \mathcal{E} \rightarrow \mathcal{E}$ that satisfies the following two properties:

1. *Identity:* If $1_{\mathcal{G}}$ is the identity element of $\mathcal{G}$, then for all $E \in \mathcal{E}$, we have $1_{\mathcal{G}} \star E = E$.
2. *Compatibility:* For any $g, h \in \mathcal{G}$ and any $E \in \mathcal{E}$, we have $(gh) \star E = g \star (h \star E)$.

We may denote a group action by using the abbreviated notation $(\mathcal{G}, \mathcal{E}, \star)$. For cryptographic purposes, we need the following properties.

Definition 15. A group action $(\mathcal{G}, \mathcal{E}, \star)$ is said to be

1. *transitive* if, for every $E_1, E_2 \in \mathcal{E}$, there exists a unique $g \in \mathcal{G}$ such that $E_2 = g \star E_1$,
2. *free* if, for all $E \in \mathcal{E}$, $E = g \star E$ implies $g = 1_{\mathcal{G}}$.

If a group action is transitive and free, it is said to be regular.

Note that if a group action is regular, then for any $E \in \mathcal{E}$, the map $f_E : g \rightarrow g \star E$ defines a bijection between $\mathcal{G}$ and $\mathcal{E}$; especially if $\mathcal{G}$ or $\mathcal{E}$ is finite, then we must have $|\mathcal{G}| = |\mathcal{E}|$.

To construct feasible cryptographic primitives from group action, we require some efficient PPT algorithms. We recall the *effective group action* framework introduced in [ADMP20].

Definition 16 (Effective Group Action [ADMP20]). A group action $(\mathcal{G}, \mathcal{E}, E_0, \star)$ is effective if the following properties are satisfied:

1. The group $\mathcal{G}$ is finite, and there exist PPT algorithms for (1) the membership testing, (2) equality testing, (3) group operations, (4) element inversions, and (5) random sampling over $\mathcal{G}$. The sampling method is required to be statistically indistinguishable from the uniform distribution over $\mathcal{G}$.
2. The set $\mathcal{E}$ is finite, and there exist PPT algorithms for (1) the membership testing and (2) generating a unique bit-string representation for every element in $\mathcal{E}$.
3. There exists a distinguished element $E_0 \in \mathcal{E}$ and its bit-string representation is publicly known.
4. There exists a PPT algorithm that given any $(g, E) \in \mathcal{G} \times \mathcal{E}$ outputs $g \star E$.

An effective group action is denoted using the abbreviated notation $(\mathcal{G}, \mathcal{E}, E_0, \star)$. Let GAGen be a PPT algorithm that takes 1^λ as input and outputs a description of effective group action $(\mathcal{G}, \mathcal{E}, E_0, \star)$. The next hardness assumption on group action is often used.

Definition 17 (Group Action Discrete Logarithm (GADL) Assumption). We say that GADL assumption holds for GAGen if for all PPT adversaries $\mathcal{A}$, it holds that

$$\text{Adv}_{\mathcal{A}, \text{GAGen}}^{\text{GADL}}(\lambda) := \Pr \left[\alpha \star E_0 = E \mid \begin{array}{l} (\mathcal{G}, \mathcal{E}, E_0, \star) \leftarrow \text{GAGen}(1^\lambda), \\ E \leftarrow \mathcal{E}, \\ \alpha \leftarrow \mathcal{A}(\mathcal{G}, \mathcal{E}, E_0, \star, E) \end{array} \right] \leq \text{negl}(\lambda).$$